

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN,
ĐHQG-HCM

KHOA TOÁN - TIN HỌC



ĐỒ ÁN CUỐI KỲ

**NHẬN DIỆN THỰC THỂ CÓ TÊN
– NAMED ENTITY
RECOGNITION**

MÔN HỌC: HỌC SÂU CHO KHOA HỌC DỮ LIỆU

Giảng viên: ThS. Hà Minh Tuấn

Nguyễn Phúc Bách	23290039
Nguyễn Đức Bảo	23280040
Hoàng Ngọc Duy	23280050
Đoàn Trần Nguyên Khang	23280064

TP. Hồ Chí Minh, tháng 6 năm 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan báo cáo bài luận với đề tài **“Phát triển và tối ưu hóa pipeline suy luận Nhận dạng Thực thể Tiếng Việt (Vietnamese NER)”** là công trình nghiên cứu tập thể do chính nhóm chúng tôi thực hiện dưới sự hướng dẫn khoa học của Giảng viên hướng dẫn.

Các số liệu thực nghiệm, kết quả đo lường độ chính xác, biểu đồ và các phân tích hệ thống được trình bày trong báo cáo này là hoàn toàn trung thực, khách quan và do nhóm trực tiếp thu thập, tổng hợp từ quá trình vận hành mã nguồn hệ thống. Công trình nghiên cứu này chưa từng được công bố dưới bất kỳ hình thức nào trong các bài luận, đồ án hay công trình khoa học của các nhóm tác giả khác trước đây.

Toàn bộ các nguồn tài liệu tham khảo, các thư viện mã nguồn kế thừa, các thuật toán nền tảng và trích dẫn khoa học sử dụng trong nội dung báo cáo đều đã được kiểm tra, xác thực và ghi chú nguồn gốc rõ ràng, minh bạch, tuân thủ nghiêm ngặt các quy định về sở hữu trí tuệ và đạo đức nghiên cứu học thuật của Nhà trường.

Chúng tôi xin hoàn toàn chịu trách nhiệm trước Giảng viên hướng dẫn và Nhà trường về tính trung thực của lời cam đoan này.

Thành phố Hồ Chí Minh, ngày ... tháng ... năm 2026

LỜI CẢM ƠN

Nhóm chúng em xin chân thành cảm ơn quý thầy cô Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM đã trang bị cho chúng em những kiến thức và kỹ năng cần thiết trong suốt quá trình học tập.

Đặc biệt, chúng em xin gửi lời cảm ơn đến **ThS. Hà Minh Tuấn** đã tận tình hướng dẫn, góp ý và hỗ trợ nhóm trong suốt quá trình thực hiện đề tài. Những nhận xét và định hướng của Thầy/Cô đã giúp nhóm hoàn thiện báo cáo cũng như nâng cao hiểu biết về bài toán Nhận dạng Thực thể Tiếng Việt.

Chúng em cũng xin cảm ơn các bạn cùng lớp đã hỗ trợ, trao đổi kiến thức và đóng góp ý kiến trong quá trình thực hiện đề tài.

Do giới hạn về thời gian và kinh nghiệm, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ quý thầy cô để có thể hoàn thiện hơn trong các nghiên cứu và dự án tiếp theo.

Nhóm xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, tháng 06 năm 2026

TÓM TẮT

Nhận dạng Thực thể Tiếng Việt (Vietnamese Named Entity Recognition - NER) là một trong những bài toán cốt lõi của Xử lý Ngôn ngữ Tự nhiên, đóng vai trò nền tảng cho việc tự động hóa trích xuất thông tin từ dữ liệu văn bản phi cấu trúc. Cùng với sự phát triển của các mô hình học sâu (Deep Learning) và đặc biệt là các mô hình ngôn ngữ lớn học chuyển tiếp (Pre-trained Language Models), độ chính xác của bài toán NER đã được nâng cao đáng kể. Tuy nhiên, thách thức lớn nhất hiện nay khi đưa các mô hình này vào môi trường công nghiệp là chi phí tài nguyên tính toán khổng lồ và độ trễ suy luận cao.

Báo cáo bài luận này trình bày toàn bộ quá trình nghiên cứu, thiết kế và phát triển một pipeline (pipeline) hoàn chỉnh nhằm giải quyết bài toán Vietnamese NER một cách toàn diện. Cụ thể, nhóm nghiên cứu đã hiện thực hóa đa dạng các kiến trúc mạng nơ-ron từ cơ bản đến nâng cao, bao gồm mạng bộ nhớ ngắn dài (LSTM, BiLSTM), kiến trúc Transformer tự xây dựng, và tích hợp mô hình ngôn ngữ tiên tiến PhoBERT. Nhằm khắc phục hạn chế của việc phân loại token độc lập, hệ thống được tích hợp lớp giải mã chuỗi Conditional Random Field (CRF) lập trình tối ưu hoàn toàn trên PyTorch, giúp mô hình nắm bắt chặt chẽ các quy tắc ràng buộc của nhãn thực thể.

Đóng góp nổi bật của bài luận nằm ở việc giải quyết bài toán tối ưu hóa tài nguyên phần cứng. Nhóm đã nghiên cứu và triển khai thành công ba kỹ thuật tiên tiến: (1) Tinh chỉnh hiệu quả tham số bằng LoRA (Low-Rank Adaptation) trên PhoBERT; (2) Chứng cất tri thức (Knowledge Distillation) để chuyển giao năng lực dự đoán từ mô hình lớn (Teacher) sang các kiến trúc nhỏ gọn (Student); và (3) Lượng tử hóa mô hình (bao gồm Post-Training Quantization và Quantization Aware Training) nhằm nén trọng số từ định dạng thực 32-bit (FP32) xuống số nguyên 8-bit (INT8). Bên cạnh đó, các kỹ thuật tăng cường dữ liệu (Data Augmentation) cũng được áp dụng nhằm gia tăng độ vững chắc cho mô hình.

Kết quả của đề tài là một hệ thống mã nguồn có cấu trúc module hóa cao, hỗ trợ huấn luyện và đánh giá chi tiết theo cả cấp độ từ (Token-level) và cấp độ thực thể (Entity-level BIO). Hệ thống không chỉ cung cấp các mô hình có độ chính xác cao mà còn đưa ra các phiên bản nén nhẹ gọn, đáp ứng tốt yêu cầu triển khai thực tế trên các thiết bị có rào cản về năng lực tính toán.

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
TÓM TẮT	iii
DANH MỤC VIẾT TẮT	xi
1 MỞ ĐẦU	1
1.1 Giới thiệu đề tài	1
1.2 Lý do chọn đề tài	2
1.2.1 Tính cấp thiết về mặt thực tiễn ứng dụng	2
1.2.2 Thách thức về mặt công nghệ và tối ưu hóa tài nguyên	2
1.2.3 Mục tiêu chuyên môn trong ngành Khoa học Dữ liệu	2
1.3 Mục tiêu đề tài	3
1.4 Phạm vi nghiên cứu	4
1.5 Bố cục của đề án	4
2 CƠ SỞ LÝ THUYẾT	6
2.1 Tổng quan về học sâu	6
2.1.1 Học sâu	6
2.2 Nhận diện thực thể có tên (Named Entity Recognition - NER)	7
2.2.1 Khái niệm	7
2.2.2 Các loại thực thể	9
2.3 Các mô hình học sâu cho NER	10
2.3.1 Bộ nhớ ngắn hạn - dài hạn (Long Short-Term Memory – LSTM)	10
2.3.1.1 Hạn chế của RNN và sự ra đời của LSTM	10
2.3.1.2 Kiến trúc mô hình LSTM	11
2.3.2 Mạng LSTM hai chiều (Bi-directional Long Short-Term Memory – BiLSTM)	13
2.3.3 Trường ngẫu nhiên có điều kiện (Conditional Random Field – CRF)	14

2.3.4	Mô hình Transformer	16
2.3.4.1	Kiến trúc Encoder–Decoder	17
2.3.4.2	Embeddings và Mã hóa vị trí	18
2.3.4.3	Cơ chế Self-Attention	19
a)	Scaled Dot-Product Attention.	19
2.3.4.4	Multi-Head Attention	19
2.3.4.5	Position-wise Feed-Forward Network	20
2.3.4.6	Add & Norm: Kết nối tắt và Chuẩn hóa lớp	20
2.3.5	Mô hình PhoBERT	21
2.4	Các kỹ thuật tối ưu hóa mô hình	21
2.4.1	Low-Rank Adaptation (LoRA)	22
2.4.1.1	Cơ sở lý thuyết và Bản chất Toán học của Phân rã Hạng thấp	22
2.4.1.2	Chiến lược khởi tạo ma trận (Initialization Strategy)	23
2.4.1.3	Vị trí tích hợp LoRA trong Kiến trúc Transformer	24
2.4.1.4	Cơ chế Hợp nhất Trọng số (Weight Merging) và Suy luận Không độ trễ	24
2.4.1.5	Phân tích So sánh Giữa LoRA và Các Phương pháp PEFT Khác	25
2.4.1.6	Ý nghĩa của LoRA đối với bài toán Nhận diện thực thể có tên (PhoNER_COVID19)	26
2.4.2	Knowledge Distillation (Chưng cất tri thức)	27
2.4.2.1	Cơ sở toán học và cơ chế chuyển giao "Tri thức tối"(Dark Knowledge)	27
2.4.2.2	Cấu trúc Hàm mất mát tổng hợp (Combined Loss Function)	28
2.4.3	Knowledge Distillation (Chưng cất tri thức)	28
2.4.3.1	Cơ sở toán học và cơ chế chuyển giao "Tri thức tối"(Dark Knowledge)	29
2.4.3.2	Cấu trúc Hàm mất mát tổng hợp (Combined Loss Function)	30
2.4.3.3	Các cơ chế điều phối học tập trong Chưng cất tri thức (Distillation Schemes)	30
a)	Offline Knowledge Distillation (Chưng cất tri thức ngoại tuyến)	31
b)	Online Knowledge Distillation (Chưng cất tri thức trực tuyến)	31
c)	Self-Knowledge Distillation (Tự chưng cất tri thức)	31
2.4.3.4	Phân loại các phương pháp Chưng cất tri thức (Taxonomy of KD)	32
a)	Response-based Knowledge Distillation	32

b)	Feature-based Knowledge Distillation	32
c)	Relation-based Knowledge Distillation	32
2.4.3.5	Ứng dụng Chứng cất tri thức trong bài toán Nhận diện Thực thể (NER)	32
a)	Đồng bộ hóa không gian chuỗi và xử lý nhân đặc biệt	33
2.4.3.6	Thách thức kỹ thuật và Chiến lược tối ưu hóa Hyperparameters	33
2.4.3.7	Kết luận	33
2.4.3.8	Phân loại các phương pháp Chứng cất tri thức (Taxonomy of KD)	34
a)	Response-based Knowledge Distillation	34
b)	Feature-based Knowledge Distillation	34
c)	Relation-based Knowledge Distillation	34
2.4.3.9	Ứng dụng Chứng cất tri thức trong bài toán Nhận diện Thực thể (NER)	34
a)	Đồng bộ hóa không gian chuỗi và xử lý nhân đặc biệt	35
2.4.3.10	Thách thức kỹ thuật và Chiến lược tối ưu hóa Hyperparameters	35
2.4.3.11	Kết luận	35
2.4.4	Post-training Quantization (PTQ)	36
2.4.4.1	Cơ sở toán học và cơ chế chuyển đổi	36
2.4.4.2	Các chiến lược lượng tử hóa trong PTQ	36
a)	Dynamic Quantization (Lượng tử hóa động)	37
b)	Static Quantization (Lượng tử hóa tĩnh)	37
2.4.4.3	Quy trình hiệu chuẩn (Calibration) - Trái tim của PTQ	37
2.4.4.4	Thách thức và giới hạn trong triển khai thực tế	38
2.4.4.5	Kết luận về PTQ trong ngữ cảnh đồ án	38
2.4.5	Quantization Aware Training (QAT): Cơ chế và Triển khai trong Hệ thống Học sâu	39
2.4.5.1	Tổng quan về Lượng tử hóa và QAT	39
2.4.5.2	Cơ sở toán học của Lượng tử hóa	39
2.4.5.3	Cơ chế hoạt động của QAT: FakeQuant và STE	39
a)	FakeQuantization (FQ)	40
b)	Straight-Through Estimator (STE)	40
2.4.5.4	Quy trình triển khai (Pipeline)	40
a)	Giai đoạn 1: Chuẩn bị (Preparation)	40
b)	Giai đoạn 2: Huấn luyện giả lập (Fake-Quant Training)	41
c)	Giai đoạn 3: Chuyển đổi (Conversion)	41

2.4.5.5	Chiến lược tối ưu hóa và Thách thức	41
a)	Lựa chọn lớp để lượng tử hóa	41
b)	Tính toán trọng số tối ưu	41
2.4.5.6	Kết luận	41
3	THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG	42
3.1	Dữ liệu và tiền xử lý	42
3.1.1	Mô tả tập dữ liệu PhoNER_COVID19	42
3.1.2	Khám phá dữ liệu	43
3.2	Quá trình huấn luyện và đánh giá	43
3.2.1	Cài đặt và thiết lập môi trường	43
3.2.2	Huấn luyện mô hình và Chiến lược tối ưu	44
3.2.2.1	Cấu hình siêu tham số và Vòng lặp huấn luyện cốt lõi	44
a)	Vòng lặp lan truyền và Cập nhật trọng số	44
b)	Điều phối Tốc độ học (Learning Rate Scheduling)	45
3.2.2.2	Cơ chế Giám sát và Dừng sớm (Early Stopping)	45
a)	Chiến lược lưu trữ điểm kiểm tra tuần hoàn (Checkpoint Management)	45
b)	Kỹ thuật dừng sớm dựa trên sự kiên nhẫn	45
3.2.2.3	Hàm mất mát tùy biến (NERLoss Configuration)	45
a)	Cấu hình Mất mát Entropy chéo (Non-CRF)	46
b)	Cấu hình Mất mát Likelihood âm (CRF-Enabled)	46
4	KẾT QUẢ VÀ THẢO LUẬN	47
4.1	Thiết lập thực nghiệm và Chỉ số đánh giá	47
4.1.1	Môi trường phần cứng và phần mềm	47
4.1.2	Siêu tham số cấu hình	48
4.1.3	Các chỉ số đo lường hiệu năng	48
4.2	Bảng so sánh hiệu năng tổng quan	49
4.3	Phân tích chuyên sâu các yếu tố ảnh hưởng	50
4.3.1	Tác động cấu trúc của lớp Conditional Random Field (CRF)	51
4.3.2	Sự vượt trội của mô hình ngôn ngữ lớn PhoBERT	52
4.3.3	Hiệu quả của kiến trúc LoRA đối với việc nén tham số	53
4.4	Đánh giá chi tiết ở cấp độ thực thể	53
4.4.1	Phân tích độ khó của các nhóm thực thể	54
4.4.1.1	Nhóm thực thể dễ nhận diện (F1-score từ 92% đến 98%)	54
4.4.1.2	Nhóm thực thể khó nhận diện (F1-score từ 60% đến 91%)	54
4.5	Phân tích các kỹ thuật nén và tối ưu hóa mô hình	55
4.5.1	Hiệu quả của Chung cất tri thức (KD)	56
4.5.2	Thảo luận về lượng tử hóa INT8: PTQ so với QAT	56

4.5.2.1	Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ)	57
4.5.2.2	Huấn luyện nhận thức lượng tử (Quantization Aware Training - QAT)	57
4.6	Phân tích lỗi định tính	58
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1	Kết luận	61
5.2	Hạn chế của đề tài	62
5.3	Hướng phát triển trong tương lai	62
	TÀI LIỆU THAM KHẢO	63

DANH MỤC BẢNG

2.1	Bảng so sánh hiệu năng giữa LoRA và các phương pháp tinh chỉnh phổ biến	26
3.1	Thống kê tổng quan về phân bố tập dữ liệu PhoNER_COVID19	43
3.2	Các siêu tham số cơ bản của hệ thống	44
4.1	Cấu hình siêu tham số huấn luyện của các lớp mô hình.	48
4.2	Bảng tổng hợp kết quả đánh giá 22 cấu hình mô hình trên tập kiểm thử (Inference trên CPU).	50
4.3	Đánh giá chi tiết trên 10 loại thực thể y tế CoNLL.	53

DANH MỤC HÌNH

2.1	Ví dụ về Named Entity Recognition (NER). [1]	7
2.2	Mô tả kiến trúc mô hình PhoNER. [2]	10
2.3	Cấu trúc các cổng trong mạng LSTM. [3]	11
2.4	Kiến trúc Bidirectional LSTM (BiLSTM). [4]	14
2.5	Kiến trúc Transformer. [5]	17
4.1	Biểu đồ so sánh BIO F1-Score giữa các cấu hình mô hình chính.	50
4.2	Đường cong loss của PhoBERT-CRF.	52
4.3	Đường cong loss của PhoBERT-LoRA-CRF.	52
4.4	So sánh đường cong loss của PhoBERT-CRF và PhoBERT-LoRA-CRF.	52
4.5	So sánh hiệu năng F1-Score chi tiết theo từng loại thực thể giữa BiLSTM-CRF và PhoBERT-LoRA-CRF.	54
4.6	Đánh đổi hiệu năng giữa độ trễ CPU, độ chính xác F1-score và dung lượng của các mô hình nén.	56
4.7	Ma trận nhầm lẫn của PhoBERT-LoRA-CRF.	58
4.8	Phân phối thực thể dự đoán của PhoBERT-LoRA-CRF.	58
4.9	Ma trận nhầm lẫn và Phân phối thực thể của PhoBERT-LoRA-CRF.	58

DANH MỤC VIẾT TẮT

Từ viết tắt	Ý nghĩa
DL	Deep Learning
ML	Machine Learning
AI	Artificial Intelligence
CV	Computer Vision
NLP	Natural Language Processing
ANN	Artificial Neural Network
GPT	Generative Pre-trained Transformer
NER	Named Entity Recognition
CRF	Conditional Random Field
RNN	Recurrent Neural Network
BPE	Byte-Pair Encoding
LSTM	Long Short-Term Memory
BiLSTM	Bidirectional Long Short-Term Memory
BERT	Bidirectional Encoder Representations from Transformers
LN	Layer Normalization
SOTA	State-of-the-art
MLM	Masked Language Modeling
NSP	Next Sentence Prediction
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training

CHƯƠNG 1

MỞ ĐẦU

1.1 Giới thiệu đề tài

Trong kỷ nguyên bùng nổ thông tin và chuyển đổi số mạnh mẽ, lượng dữ liệu văn bản phi cấu trúc (unstructured text data) được tạo ra hàng ngày trên các nền tảng mạng xã hội, trang tin tức trực tuyến, hệ thống văn bản hành chính và hồ sơ y tế là vô cùng khổng lồ. Để máy tính có thể hiểu, phân tích và tự động hóa việc khai thác nguồn tài nguyên này, lĩnh vực Xử lý Ngôn ngữ Tự nhiên đã trở thành một hướng nghiên cứu mũi nhọn trong Khoa học Dữ liệu và Trí tuệ Nhân tạo. Trong số các bài toán cốt lõi của NLP, Nhận dạng Thực thể có tên (Named Entity Recognition - NER) đóng vai trò là nền tảng, là bước tiền xử lý quyết định cho nhiều ứng dụng cấp cao như Trích xuất thông tin (Information Extraction), Tóm tắt văn bản (Text Summarization), Hệ thống hỏi đáp (Question Answering) và Xây dựng Đồ thị tri thức (Knowledge Graph).

Nhiệm vụ trọng tâm của NER là xác định các ranh giới từ hoặc cụm từ chứa thông tin định danh trong văn bản và phân loại chúng vào các danh mục thực thể đã được định nghĩa trước. Đối với ngôn ngữ tiếng Việt, việc xử lý bài toán này đối mặt với những thách thức đặc thù do đặc tính không có dấu phân tách từ rõ ràng (word segmentation), hiện tượng đa nghĩa dựa trên ngữ cảnh, và sự phong phú của cấu trúc ngữ pháp. Đặc biệt, khi áp dụng vào các lĩnh vực chuyên sâu đòi hỏi sự chính xác cao như y tế công cộng và dịch tễ học — điển hình là việc xử lý dữ liệu từ tập dữ liệu biểu diễn các thực thể dịch tễ như *PhoNER_COVID19* — bài toán NER càng trở nên phức tạp. Hệ thống không chỉ nhận diện các thực thể thông thường như tên người hay địa điểm, mà còn phải bóc tách chính xác các thực thể đặc thù như triệu chứng bệnh, phương tiện di chuyển, mã định danh bệnh nhân, hay nghề nghiệp.

Đề tài đồ án này tập trung vào việc nghiên cứu, hiện thực hóa và tối ưu hóa một hệ thống pipeline (*pipeline*) toàn diện cho bài toán Nhận dạng Thực thể Tiếng Việt. Nghiên cứu không chỉ dừng lại ở việc áp dụng các mô hình học sâu tiên tiến nhất để đạt độ chính xác

cao, mà còn tập trung giải quyết bài toán kỹ thuật thực tế: tối ưu hóa hiệu năng, giảm thiểu dung lượng tài nguyên tính toán để mô hình có khả năng triển khai thực tế trên các thiết bị đầu cuối hoặc hệ thống có cấu hình hạn chế.

1.2 Lý do chọn đề tài

Việc lựa chọn nghiên cứu và phát triển đề tài **“Xây dựng và tối ưu hóa pipeline Nhận dạng Thực thể Tiếng Việt (Vietnamese NER) ứng dụng các kỹ thuật Học sâu tiên tiến”** xuất phát từ ba động lực chính sau đây:

1.2.1 Tính cấp thiết về mặt thực tiễn ứng dụng

Trong các tình huống khẩn cấp về y tế hoặc quản lý dữ liệu xã hội, việc xử lý thủ công hàng ngàn văn bản, báo cáo không cấu trúc để trích xuất thông tin dịch tễ là bất khả thi về mặt thời gian và nhân lực. Một hệ thống NER tự động có khả năng nhận diện chính xác các thực thể như SYMPTOM_AND_DISEASE (Triệu chứng và bệnh), TRANSPORTATION (Phương tiện di chuyển), hoặc LOCATION (Địa điểm) sẽ giúp các nhà phân tích dữ liệu nhanh chóng xây dựng được bức tranh toàn cảnh, hỗ trợ đắc lực cho công tác ra quyết định. Do đó, việc xây dựng một công cụ NER chuyên biệt cho tiếng Việt mang giá trị thực tiễn rất lớn.

1.2.2 Thách thức về mặt công nghệ và tối ưu hóa tài nguyên

Hiện nay, các mô hình ngôn ngữ lớn dựa trên kiến trúc Transformer tiền huấn luyện (Pre-trained Language Models) như PhoBERT đã thiết lập những kỷ lục mới về độ chính xác (State-of-the-Art) cho các bài toán NLP tiếng Việt. Tuy nhiên, rào cản lớn nhất của các mô hình này là kích thước tham số khổng lồ, đòi hỏi năng lực tính toán cực kỳ cao (GPU/TPU) và độ trễ suy luận (inference latency) lớn. Trong môi trường công nghiệp hoặc triển khai trên thiết bị phần cứng hạn chế, việc chạy trực tiếp các mô hình float32 nguyên bản là không khả thi. Điều này đặt ra nhu cầu cấp thiết phải nghiên cứu các giải pháp nén và tối ưu hóa mô hình như Tinh chỉnh hiệu quả tham số (Parameter-Efficient Fine-Tuning - PEFT với LoRA), Chưng cất tri thức (Knowledge Distillation), và Lượng tử hóa trọng số (Quantization) để cân bằng giữa bài toán giữ vững độ chính xác và tăng tốc độ xử lý.

1.2.3 Mục tiêu chuyên môn trong ngành Khoa học Dữ liệu

Là những sinh viên ngành Khoa học Dữ liệu, việc thực hiện đề tài này là cơ hội để hệ thống hóa, làm chủ và kết hợp các khối kiến thức toán học tối ưu, lý thuyết xác suất thống kê với các kỹ năng lập trình hệ thống học sâu tiên tiến. Đề tài đòi hỏi việc xây

dựng cấu trúc mã nguồn một cách khoa học, module hóa từ khâu tăng cường dữ liệu (*Data Augmentation*), lập trình các kiến trúc mạng từ cơ bản đến nâng cao (LSTM, Transformer, BERT), cho đến khâu can thiệp sâu vào toán học của lớp giải mã chuỗi Conditional Random Field (CRF) và các thuật toán nén mô hình ở mức độ thư viện phần cứng.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đề án này là thiết kế và phát triển thành công một hệ thống mã nguồn tối ưu, có cấu trúc phân tầng rõ ràng, phục vụ cho việc huấn luyện, đánh giá, chưng cất và nén các mô hình nhận dạng thực thể tiếng Việt. Để đạt được mục tiêu tổng

quát, các mục tiêu cụ thể được thiết lập bao gồm:

1. **Hiện thực hóa đa dạng kiến trúc học sâu:** Xây dựng từ đầu (from scratch) các mô hình nền tảng bao gồm mạng bộ nhớ ngắn dài đơn hướng (LSTM), mạng bộ nhớ ngắn dài hai hướng (BiLSTM), và kiến trúc khối mã hóa Native Transformer để làm mô hình cơ sở so sánh.
2. **Tích hợp và tinh chỉnh mô hình ngôn ngữ lớn:** Tích hợp thành công mô hình học chuyển tiếp PhoBERT; áp dụng kiến trúc **LoRA** (Low-Rank Adaptation) để đóng băng phần lớn trọng số cốt lõi và chỉ cập nhật các ma trận hạng thấp, nhằm giảm thiểu tối đa chi phí bộ nhớ đồ họa khi huấn luyện.
3. **Xây dựng lớp giải mã chuỗi tối ưu:** Tự hiện thực hóa lớp cấu trúc tuyến tính **Conditional Random Field (CRF)** hoàn toàn bằng các toán tử vectơ của PyTorch, giúp mô hình học được xác suất chuyển cấu trúc nhãn chuỗi thay vì phân loại độc lập từng token, từ đó tối ưu hóa độ chính xác của ranh giới thực thể.
4. **Nén và tăng tốc mô hình (Model Compression):**
 - Triển khai kỹ thuật **Chưng cất tri thức (Knowledge Distillation)** để chuyển giao tri thức từ mô hình PhoBERT Teacher (độ chính xác cao nhưng nặng) sang các mô hình Student nhỏ gọn hơn (LSTM hoặc Transformer), giúp mô hình nhỏ đạt hiệu suất vượt bậc.
 - Triển khai kỹ thuật **Lượng tử hóa Post-Training (PTQ)** và **Huấn luyện nhận thức lượng tử (Quantization Aware Training - QAT)** để chuyển đổi định dạng lưu trữ trọng số từ dạng số thực 32-bit (FP32) sang số nguyên 8-bit (INT8), tối ưu hóa dung lượng lưu trữ vật lý và tốc độ suy luận trên CPU.

5. **Xây dựng hệ thống đánh giá chặt chẽ:** Thiết lập module đo lường hiệu năng toàn diện ở cả cấp độ Token-level (Sử dụng Sklearn metrics) và Entity-level (Sử dụng biểu diễn BIO thông qua Seqeval), kèm theo các công cụ trực quan hóa đồ thị loss, ma trận nhầm lẫn (Confusion Matrix) phục vụ cho quá trình phân tích lỗi hệ thống.

1.4 Phạm vi nghiên cứu

Nhằm đảm bảo tính tập trung và độ sâu về mặt học thuật trong giới hạn thời gian và tài nguyên phần cứng cho phép, phạm vi nghiên cứu của đề án được xác định cụ thể như sau:

- **Đối tượng dữ liệu:** Nghiên cứu tập trung hoàn toàn vào dữ liệu văn bản tiếng Việt. Tập dữ liệu thực nghiệm được chuẩn hóa theo định dạng CoNLL, bao gồm tổ hợp 20 nhãn thực thể dựa trên quy tắc BIO.
- **Kỹ thuật tiền xử lý ngôn ngữ:** Giới hạn xử lý ngôn ngữ ở mức độ cụm từ phụ (*subword-level tokenization*) bằng cách đồng bộ hóa cấu trúc Byte-Pair Encoding (BPE) của nền tảng Hugging Face kết hợp công cụ tách từ PyVi để tối ưu hóa đặc trưng ngữ nghĩa tiếng Việt.
- **Giải pháp tăng cường dữ liệu:** Áp dụng thuật toán tăng cường dữ liệu tại chỗ thông qua cơ chế thay thế thực thể đồng dạng (*Entity Replacement Augmentation*) nhằm cải thiện chất lượng tập huấn luyện đối với các nhãn hiếm, không can thiệp vào các phương pháp sinh văn bản tự sinh bằng LLMs tạo sinh.
- **Môi trường thực nghiệm và công nghệ:** Toàn bộ hệ thống pipeline được phát triển trên ngôn ngữ lập trình Python, sử dụng thư viện cốt lõi PyTorch và các công cụ hỗ trợ liên quan. Thử nghiệm huấn luyện và đánh giá được thực hiện linh hoạt trên các kiến trúc phần cứng bao gồm CPU, đồ họa NVIDIA (với thư viện tăng tốc CUDA) và chip Apple Silicon (với MPS backend), tích hợp cơ chế tự động chuyển đổi thiết bị khi phát hiện xung đột tính toán lớp CRF.

1.5 Bố cục của đề án

Nội dung cuốn đề án được cấu trúc thành 5 chương chính như sau:

Chương 1: Mở đầu Giới thiệu tổng quan về bài toán Nhận dạng Thực thể Tiếng Việt, tính cấp thiết của đề tài, các mục tiêu cụ thể cần đạt được và giới hạn phạm vi nội dung nghiên cứu.

Chương 2: Cơ sở lý thuyết Trình bày nền tảng toán học và lý thuyết mạng nơ-ron của các kiến trúc LSTM, BiLSTM, cơ chế Attention, kiến trúc Transformer và mô hình PhoBERT. Đi sâu vào nguyên lý hoạt động của thuật toán Viterbi và mô hình chuỗi CRF, cùng toán học của các phương pháp nén mô hình như LoRA, Knowledge Distillation và Quantization.

Chương 3: Thiết kế và Hiện thực hóa hệ thống Pipeline Chi tiết hóa kiến trúc hệ thống, sơ đồ luồng dữ liệu và thiết kế module của mã nguồn. Mô tả cách thức lập trình và triển khai cụ thể các module tiền xử lý dữ liệu, xây dựng mô hình, bộ huấn luyện đặc thù cho bài toán chứng cất và lượng tử hóa.

Chương 4: Kết quả thực nghiệm và Phân tích Trình bày các kịch bản thử nghiệm, bảng thông số so sánh hiệu năng (Loss, Accuracy, F1-score) giữa các mô hình trước và sau khi tối ưu hóa. Phân tích chi tiết kích thước mô hình, tốc độ suy luận, ma trận nhầm lẫn và thực hiện phân tích lỗi định tính trên các câu văn bản cụ thể.

Chương 5: Kết luận và Hướng phát triển Tổng kết lại các kết quả đạt được của khóa luận, nhìn nhận các hạn chế còn tồn tại và đề xuất các hướng nghiên cứu mở rộng trong tương lai nhằm nâng cao chất lượng hệ thống.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về học sâu

2.1.1 Học sâu

Học sâu hay còn được biết đến rộng rãi là Deep Learning, là một lĩnh vực con của học máy – Machine Learning, tập trung vào việc phát triển các mô hình có khả năng tự động học và trích xuất đặc trưng từ dữ liệu thông qua các mạng nơ-ron nhân tạo nhiều tầng (Artificial Neural Networks) [6, 7]. Học sâu được lấy cảm hứng từ cấu trúc và cơ chế hoạt động của bộ não con người, trong đó các nơ-ron sinh học được mô phỏng bởi các nơ-ron nhân tạo liên kết với nhau thành nhiều lớp. Nhờ khả năng học các biểu diễn dữ liệu ở nhiều mức độ trừu tượng khác nhau, những quan hệ ẩn sâu bên trong dữ liệu.

Kiến trúc của mô hình học sâu được xây dựng dựa trên mạng nơ-ron nhân tạo ANN. Một mô hình học sâu thường bao gồm nhiều lớp nơ-ron được sắp xếp theo thứ tự từ lớp đầu vào (Input Layer), các lớp ẩn (Hidden Layers) đến lớp đầu ra (Output Layer). Mỗi nơ-ron trong một lớp được kết nối với các nơ-ron ở lớp kế tiếp thông qua các trọng số (weights), cho phép mô hình học và biểu diễn các đặc trưng phức tạp từ dữ liệu [7].

Trong quá trình huấn luyện, dữ liệu được truyền từ lớp đầu vào qua các lớp ẩn thông qua cơ chế lan truyền xuôi (Forward Propagation). Tại mỗi nơ-ron, dữ liệu được biến đổi bằng phép tính tuyến tính kết hợp với hàm kích hoạt (Activation Function) nhằm đưa tính phi tuyến vào mô hình. Kết quả dự đoán ở lớp đầu ra được so sánh với nhãn thực tế thông qua hàm mất mát (Loss Function). Sau đó, thuật toán lan truyền ngược (Backpropagation) [8] cùng với các thuật toán tối ưu hóa như SGD hoặc Adam [9] sẽ cập nhật các trọng số để giảm sai số dự đoán.

Điểm khác biệt giữa học sâu và các mạng nơ-ron truyền thống là số lượng lớp ẩn lớn hơn đáng kể, cho phép mô hình tự động học các biểu diễn đặc trưng ở nhiều mức độ trừu tượng khác nhau [10]. Nhờ đó, học sâu đạt hiệu quả cao trong các bài toán phức tạp như

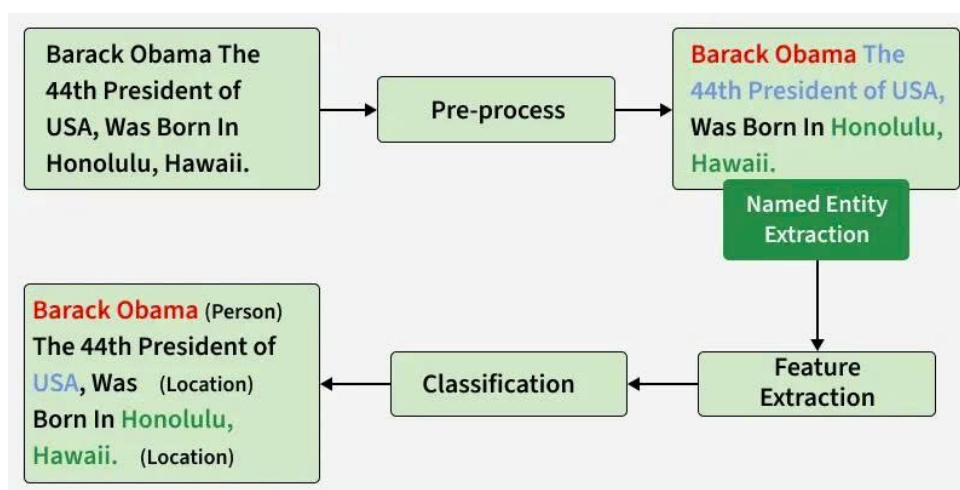
nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên và nhận dạng giọng nói.

Có thể phân học sâu thành năm loại chính: Thị giác máy tính - Computer Vision, Nhận dạng giọng nói - Voice Recognition, Xử lý ngôn ngữ tự nhiên - Natural Language Processing, Công cụ đề xuất - Recommendation Systems và AI tạo sinh - Generative AI. Một số ứng dụng nổi bật có thể kể đến hiện nay có thể kể đến như nhận diện khuôn mặt và phát hiện vật thể trong ảnh thuộc lĩnh vực thị giác máy tính; trợ lý ảo chuyển đổi giọng nói thành văn bản trong nhận dạng giọng nói; dịch máy, phân tích cảm xúc và chatbot thông minh trong xử lý ngôn ngữ tự nhiên; hệ thống gợi ý phim, âm nhạc và sản phẩm trên các nền tảng thương mại điện tử trong công cụ đề xuất; hay các mô hình tạo văn bản, hình ảnh và video như ChatGPT, Gemini, Midjourney và Sora trong lĩnh vực AI tạo sinh. Những ứng dụng này đang được triển khai rộng rãi trong nhiều lĩnh vực như y tế, giáo dục, tài chính, thương mại điện tử và giải trí, góp phần nâng cao hiệu quả công việc cũng như trải nghiệm của người dùng.

2.2 Nhận diện thực thể có tên (Named Entity Recognition - NER)

2.2.1 Khái niệm

Nhận diện thực thể có tên - NER là một ứng dụng trong NLP, tập trung vào việc xác định/ phân loại các thông tin quan trọng được gọi là các thực thể trong văn bản [11, 12]. Những thực thể này có thể là tên một người, địa chỉ hoặc một tổ chức... NER giúp chuyển đổi những văn bản không có cấu trúc thành các thông tin có cấu trúc để phục vụ cho các công việc như tóm tắt văn bản, tạo biểu đồ tri thức và trả lời câu hỏi.



Hình 2.1: Ví dụ về Named Entity Recognition (NER). [1]

Có nhiều phương pháp khác nhau được sử dụng trong các bài toán về NER, bao gồm:

- **Phương pháp dựa trên từ điển – Lexicon-based Method:**

Phương pháp này sử dụng một từ điển chứa các tên thực thể đã được biết trước cấu trúc. Quá trình nhận dạng được thực hiện bằng cách kiểm tra xem các từ hoặc cụm từ trong văn bản có xuất hiện trong từ điển hay không [11]. Tuy nhiên, cách tiếp cận này ít được sử dụng vì từ điển cần cập nhật thường xuyên và kích thước của dữ liệu - corpus sẽ có thể tăng theo thời gian. Ví dụ, các từ lóng của giới trẻ hiện nay xuất hiện với tần suất ngày càng cao, đòi hỏi từ điển phải được cập nhật liên tục.

- **Phương pháp dựa trên luật – Rule-based Method:**

Phương pháp này sử dụng một tập hợp các luật được định nghĩa trước để trích xuất thông tin từ văn bản [13]. Các luật thường dựa trên mẫu (pattern) và ngữ cảnh (context):

- **Luật dựa trên mẫu (Pattern-based Rules):** Tập trung vào cấu trúc và hình thức của từ, chẳng hạn như đặc điểm hình thái học, chữ viết hoa, tiền tố hoặc hậu tố.
- **Luật dựa trên ngữ cảnh (Context-based Rules):** Xem xét các từ xung quanh hoặc ngữ cảnh mà một từ xuất hiện trong tài liệu.

Việc kết hợp cả hai loại luật này giúp nâng cao độ chính xác của quá trình nhận dạng thực thể.

- **Phương pháp dựa trên học máy – Machine Learning-Based Method:**

- **Phân loại đa lớp (Multi-Class Classification):** Mô hình được huấn luyện trên tập dữ liệu đã được gán nhãn, trong đó mỗi thực thể thuộc về một lớp xác định. Ngoài việc học nhãn, mô hình còn cần hiểu được ngữ cảnh của từ, khiến đây trở thành một nhiệm vụ tương đối khó đối với các thuật toán học máy truyền thống.
- **Conditional Random Field:** CRF là một mô hình xác suất thường được sử dụng trong các bài toán gán nhãn chuỗi [14]. Mô hình này xem xét mối quan hệ giữa các từ trong câu cũng như sự phụ thuộc giữa các nhãn, từ đó giúp dự đoán thực thể chính xác hơn. CRF được triển khai trong nhiều thư viện xử lý ngôn ngữ tự nhiên như NLTK [15] và các bộ gán nhãn từ loại (POS Tagger).

- **Phương pháp dựa trên học sâu – Deep Learning-Based Method:**

Các mô hình học sâu hiện là hướng tiếp cận phổ biến nhất trong NER nhờ khả năng tự động học đặc trưng từ dữ liệu [12, 16].

- **Word Embeddings:** Các từ được chuyển đổi thành các vector số học có khả năng biểu diễn ý nghĩa ngữ nghĩa và ngữ cảnh của từ trong văn bản [17, 18].
- **Học đặc trưng tự động:** Mô hình học sâu có thể tự động khám phá các đặc trưng phức tạp mà không cần thiết kế thủ công.
- **Độ chính xác cao:** Các mô hình học sâu thường đạt hiệu suất vượt trội trên các tập dữ liệu lớn và đa dạng.

2.2.2 Các loại thực thể

Tập dữ liệu được sử dụng là **PhoNER_COVID19** [2] dùng để nhận diện các thực thể tiếng Việt có liên quan đến COVID-19, bộ dữ liệu chứa 35.000 thực thể và khoảng hơn 10.000 câu. Bộ dữ liệu được cung cấp chứ 10 loại thực thể, mục đích để trích xuất các thông tin quan trọng liên quan đến các bệnh nhân mắc COVID-19, từ đó có thể áp dụng vào các ứng dụng hạ nguồn – downstream applications. Ngoài ra có thể ứng dụng cho các dịch bệnh khác trong hiện tại và trong tương lai chứ không chỉ mỗi đại dịch COVID-19.

10 loại thực thể bao gồm:

- **PATIENT_ID:** Mã định danh duy nhất của bệnh nhân COVID-19.
- **PERSON_NAME:** Tên của bệnh nhân hoặc người có tiếp xúc với bệnh nhân.
- **AGE:** Tuổi của bệnh nhân hoặc người có tiếp xúc với bệnh nhân.
- **GENDER:** Giới tính của bệnh nhân hoặc người có tiếp xúc với bệnh nhân.
- **OCCUPATION:** Nghề nghiệp của bệnh nhân hoặc người có tiếp xúc với bệnh nhân.
- **LOCATION:** Địa điểm hoặc vị trí mà bệnh nhân đã từng xuất hiện.
- **ORGANIZATION:** Tên các tổ chức có liên quan đến bệnh nhân, chẳng hạn như công ty, cơ quan nhà nước hoặc các tổ chức khác.
- **SYMPTOM&DISEASE:** Các triệu chứng hoặc bệnh lý mà bệnh nhân mắc phải.
- **TRANSPORTATION:** Phương tiện di chuyển mà bệnh nhân sử dụng, thường được biểu diễn bằng số hiệu chuyến bay hoặc biển số xe.
- **DATE:** Các mốc thời gian hoặc ngày tháng xuất hiện trong văn bản.

Label	Definition
PATIENT_ID	Unique identifier of a COVID-19 patient in Vietnam. An PATIENT_ID annotation over “X” refers to as the X th patient having COVID-19 in Vietnam.
PERSON_NAME	Name of a patient or person who comes into contact with a patient.
AGE	Age of a patient or person who comes into contact with a patient.
GENDER	Gender of a patient or person who comes into contact with a patient.
OCCUPATION	Job of a patient or person who comes into contact with a patient.
LOCATION	Locations/places that a patient was presented at.
ORGANIZATION	Organizations related to a patient, e.g. company, government organization, and the like, with structures and their own functions.
SYMPTOM&DISEASE	Symptoms that a patient experiences, and diseases that a patient had prior to COVID-19 or complications that usually appear in death reports.
TRANSPORTATION	Means of transportation that a patient used. Here, we only tag the specific identifier of vehicles, e.g. flight numbers and bus/car plates.
DATE	Any date that appears in the sentence.

Table 1: Definitions of entity types in our annotation guidelines. We do not annotate nested entities.

Hình 2.2: Mô tả kiến trúc mô hình PhoNER. [2]

2.3 Các mô hình học sâu cho NER

2.3.1 Bộ nhớ ngắn hạn - dài hạn (Long Short-Term Memory – LSTM)

2.3.1.1 Hạn chế của RNN và sự ra đời của LSTM

Mạng nơ-ron hồi quy RNN được thiết kế nhằm xử lý dữ liệu tuần tự bằng cách duy trì trạng thái ẩn (hidden state) để lưu giữ thông tin từ các bước thời gian trước đó [19]. Tuy nhiên, RNN truyền thống gặp phải một số hạn chế khi xử lý các chuỗi có độ dài lớn. Vấn đề về tiêu biến gradient – vanishing gradient và bùng nổ gradient – exploding gradient trong quá trình lan truyền ngược qua thời gian [20, 21].

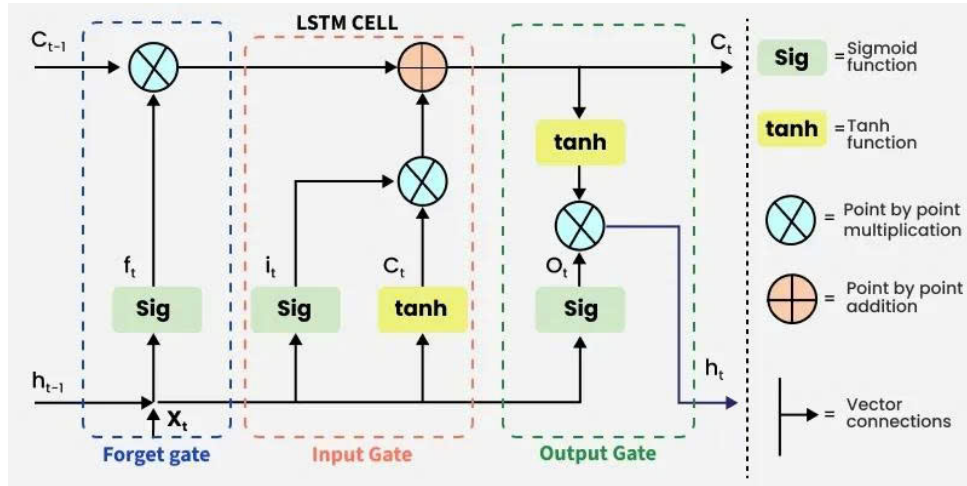
Hiện tượng tiêu biến gradient xảy ra khi giá trị gradient trở nên rất nhỏ sau nhiều bước lan truyền ngược, khiến các trọng số ở những bước thời gian xa gần như không được cập nhật. Do đó, mô hình gặp khó khăn trong việc học và ghi nhớ các phụ thuộc dài hạn (long-term dependencies) trong chuỗi dữ liệu [20]. Ngược lại, hiện tượng bùng nổ gradient xảy ra khi gradient tăng quá lớn, dẫn đến quá trình huấn luyện không ổn định và khó hội tụ.

Do hạn chế về khả năng ghi nhớ thông tin dài hạn, RNN thường không đạt được hiệu quả cao trong các tác vụ xử lý ngôn ngữ tự nhiên, chẳng hạn ý nghĩa của một từ đôi khi phụ thuộc và các từ xuất hiện từ lâu trước đó trong câu hoặc đoạn văn.

Để khắc phục các vấn đề trên, kiến trúc Long Short-Term Memory (LSTM) đã được đề xuất bởi Sepp Hochreiter và Jürgen Schmidhuber vào năm 1997 [22]. LSTM bổ sung một

cơ chế bộ nhớ (memory cell) cùng các cổng điều khiển (gates) nhằm kiểm soát luồng thông tin được lưu giữ, cập nhật hoặc loại bỏ. Nhờ đó, LSTM có khả năng ghi nhớ thông tin trong khoảng thời gian dài hơn và giảm đáng kể ảnh hưởng của hiện tượng tiêu biến gradient, giúp mô hình học hiệu quả hơn trên các dữ liệu tuần tự có phụ thuộc dài hạn.

2.3.1.2 Kiến trúc mô hình LSTM



Hình 2.3: Cấu trúc các cổng trong mạng LSTM. [3]

Kiến trúc của LSTM được xây dựng dựa trên một đơn vị bộ nhớ (memory cell) và ba cổng điều khiển chính: cổng quên (forget gate) [23] – trả lời câu hỏi "**Nên quên gì?**", cổng vào (input gate) – trả lời câu hỏi "**Nên ghi gì?**" và cổng ra (output gate) – trả lời câu hỏi "**Nên xuất gì?**".

Cổng quên (Forget gate)

Cổng quên dùng để quyết định thông tin nào từ trạng thái bộ nhớ trước đó sẽ được giữ lại hoặc loại bỏ. Nó sử dụng đầu vào hiện tại x_t và trạng thái ẩn trước đó h_{t-1} , truyền qua hàm **sigmoid** để tạo ra giá trị nằm trong khoảng $(0, 1)$.

- Giá trị gần 0: loại bỏ thông tin.
- Giá trị gần 1: giữ lại thông tin.
- Giúp loại bỏ các thông tin không cần thiết.
- Kiểm soát quá trình ghi nhớ trong LSTM.

Công thức của cổng quên:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.1)$$

Trong đó:

- f_t là vector giá trị của cổng quên tại thời điểm t
- σ là hàm sigmoid
- W_f, b_f lần lượt là ma trận trọng số và bias của cổng quên
- h_{t-1} là hidden state tại thời điểm trước
- x_t là input tại thời điểm hiện tại

Cổng vào (Input gate)

Cổng vào quyết định lượng thông tin mới sẽ được cập nhật vào bộ nhớ.

- Đầu tiên, thông tin được điều chỉnh bằng hàm sigmoid để lọc ra những giá trị cần ghi nhớ, tương tự như cổng quên, dựa trên các đầu vào x_t và h_{t-1} .
- Sau đó, một vector ứng viên cho trạng thái bộ nhớ mới được tạo ra bằng hàm tanh, cho ra các giá trị trong khoảng từ -1 đến +1, chứa toàn bộ thông tin tiềm năng được suy ra từ x_t và h_{t-1} .
- Cuối cùng, vector này được nhân với các giá trị đã được điều chỉnh bởi sigmoid để thu được phần thông tin hữu ích cần được cập nhật vào trạng thái bộ nhớ.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.3)$$

Trong đó:

- i_t là cổng vào
- $\tilde{C}_t$ là giá trị trạng thái bộ nhớ ứng viên

Tiếp theo, cập nhật trạng thái bộ nhớ bằng cách nhân trạng thái bộ nhớ trước đó với f_t , việc này giúp loại bỏ thông tin dư thừa từ sớm. Tiếp đến LSTM sẽ nhân $i_t \odot \tilde{C}_t$ để biểu diễn bao nhiêu thông tin sẽ được cập nhật. Cuối cùng lấy tổng 2 phép nhân element-wise lại ta được trạng thái bộ nhớ mới.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (2.4)$$

Trong đó:

- C_t là cell state tại thời điểm t
- $\odot$ là phép nhân từng phần tử (element-wise multiplication)

Cổng ra (Output gate)

Cổng ra quyết định thông tin nào từ trạng thái bộ nhớ hiện tại sẽ được đưa ra làm trạng thái ẩn (hidden state) tại bước thời gian hiện tại. Nó sử dụng trạng thái ẩn trước đó h_{t-1} và đầu vào hiện tại x_t , sau đó áp dụng hàm sigmoid để kiểm soát luồng thông tin đầu ra.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.5)$$

Tiếp theo, trạng thái bộ nhớ hiện tại C_t được đưa qua hàm kích hoạt tanh để chuẩn hóa giá trị về khoảng từ -1 đến +1. Cuối cùng, trạng thái đã biến đổi này được nhân theo từng phần tử với o_t để tạo ra trạng thái ẩn h_t :

$$h_t = o_t \odot \tanh(C_t) \quad (2.6)$$

Tóm lại, LSTM sử dụng cơ chế cổng để kiểm soát dòng thông tin, giúp mô hình có khả năng ghi nhớ thông tin quan trọng trong thời gian dài và loại bỏ các thông tin không cần thiết, từ đó cải thiện hiệu quả học trên dữ liệu chuỗi [22].

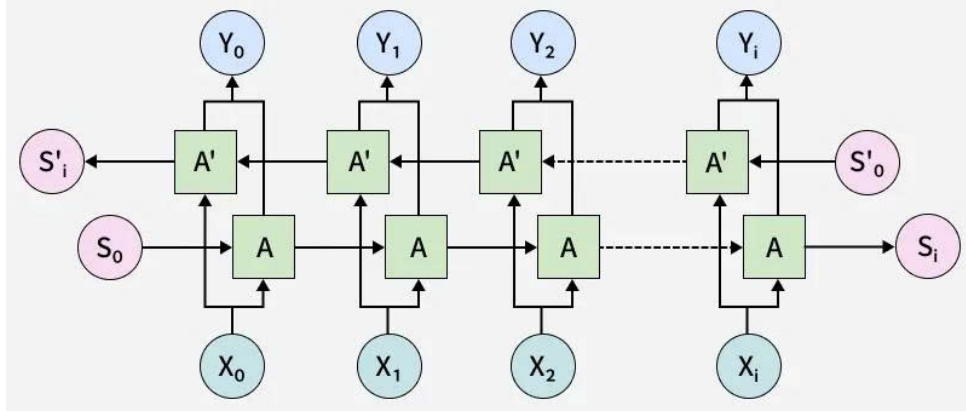
2.3.2 Mạng LSTM hai chiều (Bi-directional Long Short-Term Memory – BiLSTM)

BiLSTM là một biến thể mở rộng của mạng LSTM, trong đó chuỗi dữ liệu được xử lý đồng thời theo cả hai hướng: từ đầu đến cuối (forward) và từ cuối về đầu (backward) [24, 25]. Nhờ đó, mô hình có khả năng khai thác không chỉ thông tin từ quá khứ mà còn cả ngữ cảnh tương lai của mỗi phần tử trong chuỗi.

Một số đặc điểm nổi bật của BiLSTM bao gồm:

- Xử lý chuỗi theo cả hai chiều tiến và lùi.
- Khai thác đồng thời thông tin ngữ cảnh quá khứ và tương lai.
- Hiệu quả hơn LSTM một chiều trong các bài toán yêu cầu hiểu ngữ cảnh toàn diện.
- Được ứng dụng rộng rãi trong các lĩnh vực NLP, nhận dạng giọng nói và các bài toán phân tích dữ liệu tuần tự [26, 16, 27].

Kiến trúc của BiLSTM:



Hình 2.4: Kiến trúc Bidirectional LSTM (BiLSTM). [4]

Mạng BiLSTM bao gồm hai lớp LSTM hoạt động độc lập:

- **Forward LSTM:** xử lý chuỗi theo chiều từ đầu đến cuối.
- **Backward LSTM:** xử lý chuỗi theo chiều từ cuối về đầu.

Đầu ra của hai mạng LSTM này sau đó được kết hợp để tạo thành đầu ra cuối cùng của mô hình. Đầu ra tại thời điểm t được xác định như sau.

$$p_t = p_t^f + p_t^b \quad (2.7)$$

Trong đó:

- p_t : vector xác suất đầu ra cuối cùng tại thời điểm t .
- p_t^f : vector xác suất được sinh ra từ mạng Forward LSTM.
- p_t^b : vector xác suất được sinh ra từ mạng Backward LSTM.

2.3.3 Trường ngẫu nhiên có điều kiện (Conditional Random Field – CRF)

Trong bài toán gán nhãn chuỗi (*sequence labeling*) nói chung và nhận diện thực thể có tên (Named Entity Recognition – NER) nói riêng, các nhãn đầu ra không độc lập với nhau mà tuân theo những ràng buộc ngữ pháp chặt chẽ. Chẳng hạn, theo quy ước gán nhãn BIO [28], một nhãn I-LOC không thể đứng ngay sau

B-PER, hay I-PER không được phép xuất hiện ở đầu câu. Nếu tại mỗi vị trí, mô hình quyết định nhãn một cách độc lập (chẳng hạn dùng hàm kích hoạt **Softmax** cho đầu ra của một mô hình học sâu), những ràng buộc này hoàn toàn bị bỏ qua, dẫn đến chuỗi

nhân dự đoán thiếu nhất quán. Để khắc phục hạn chế đó, **Lafferty** và cộng sự đã đề xuất *Trường ngẫu nhiên có điều kiện*

(Conditional Random Field – CRF) [14], một mô hình đồ thị xác suất có khả năng mô hình hóa trực tiếp phân phối có điều kiện của toàn bộ chuỗi nhãn dựa trên chuỗi quan sát.

Định nghĩa hình thức

Cho chuỗi quan sát đầu vào $\mathbf{x} = (x_1, x_2, \dots, x_n)$ và chuỗi nhãn tương ứng $\mathbf{y} = (y_1, y_2, \dots, y_n)$ với $y_i \in \mathcal{Y}$ ($\mathcal{Y}$ là tập nhãn). Mô hình CRF tuyến tính (*linear-chain CRF*) [14, 29] định nghĩa xác suất

có điều kiện của $\mathbf{y}$ khi biết $\mathbf{x}$ dưới dạng:

$$P(\mathbf{y} \mid \mathbf{x}) = \frac{e^{s(\mathbf{x}, \mathbf{y})}}{\sum_{\mathbf{y}' \in \mathcal{Y}^n} e^{s(\mathbf{x}, \mathbf{y}')}}, \quad (2.8)$$

trong đó hàm điểm $s(\mathbf{x}, \mathbf{y})$ được phân rã thành tổng của hai thành phần:

$$s(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n E_{i, y_i} + \sum_{i=1}^{n-1} T_{y_i, y_{i+1}}. \quad (2.9)$$

Ở đây, E_{i, y_i} là điểm phát xạ (*emission score*) – điểm số tại vị trí thứ i nhận nhãn y_i , thường được lấy từ đầu ra của tầng mạng phía dưới; còn $T_{y_i, y_{i+1}}$ là điểm chuyển trạng thái (*transition score*) từ nhãn y_i sang nhãn y_{i+1} , được học cùng với các tham số khác của mô hình. Mẫu số trong (2.8) đóng vai trò hàm chuẩn hóa (*partition function*) $Z(\mathbf{x})$, lấy tổng trên toàn bộ chuỗi nhãn khả dĩ.

Huấn luyện

Tham số của CRF được tối ưu bằng cách **tối thiểu hóa hàm mất mát negative log-likelihood**:

$$\mathcal{L}_{\text{CRF}} = -\log P(\mathbf{y}^* \mid \mathbf{x}) = \log Z(\mathbf{x}) - s(\mathbf{x}, \mathbf{y}^*), \quad (2.10)$$

với $\mathbf{y}^*$ là chuỗi nhãn đúng (*gold sequence*). Việc tính trực tiếp $Z(\mathbf{x})$ là không khả thi do số chuỗi nhãn tăng theo cấp số mũ $|\mathcal{Y}|^n$; tuy nhiên, nhờ tính chất Markov bậc một của linear-chain CRF, $Z(\mathbf{x})$ có thể được tính hiệu quả bằng thuật toán *forward* với độ phức tạp

$$\mathcal{O}(n \cdot |\mathcal{Y}|^2) \text{ [29].}$$

Suy luận bằng giải thuật Viterbi

Ở giai đoạn suy luận, mục tiêu là tìm chuỗi nhãn có xác suất hậu nghiệm cao nhất:

$$\mathbf{y}^* = \arg \max_{\mathbf{y} \in \mathcal{Y}^n} P(\mathbf{y} \mid \mathbf{x}) = \arg \max_{\mathbf{y} \in \mathcal{Y}^n} s(\mathbf{x}, \mathbf{y}). \quad (2.11)$$

Do không gian tìm kiếm có kích thước $|\mathcal{Y}|^n$, việc duyệt vét cạn - Brute Force là không khả thi với những câu có độ dài thông thường. Để giải bài toán này một cách hiệu quả, ta sử dụng giải thuật **Viterbi** [30] – là một ứng dụng của quy hoạch động được áp dụng rộng rãi cho các mô hình Markov ẩn và CRF tuyến tính.

Ý tưởng cốt lõi của Viterbi là tại mỗi bước thời gian i và với mỗi nhãn $y \in \mathcal{Y}$, ta chỉ cần lưu lại điểm số lớn nhất của tất cả các chuỗi nhãn có độ dài i kết thúc bằng y , ký hiệu là $\delta_i(y)$. Đại lượng này được tính theo công thức truy hồi:

$$\delta_1(y) = P_{1,y}, \quad (2.12)$$

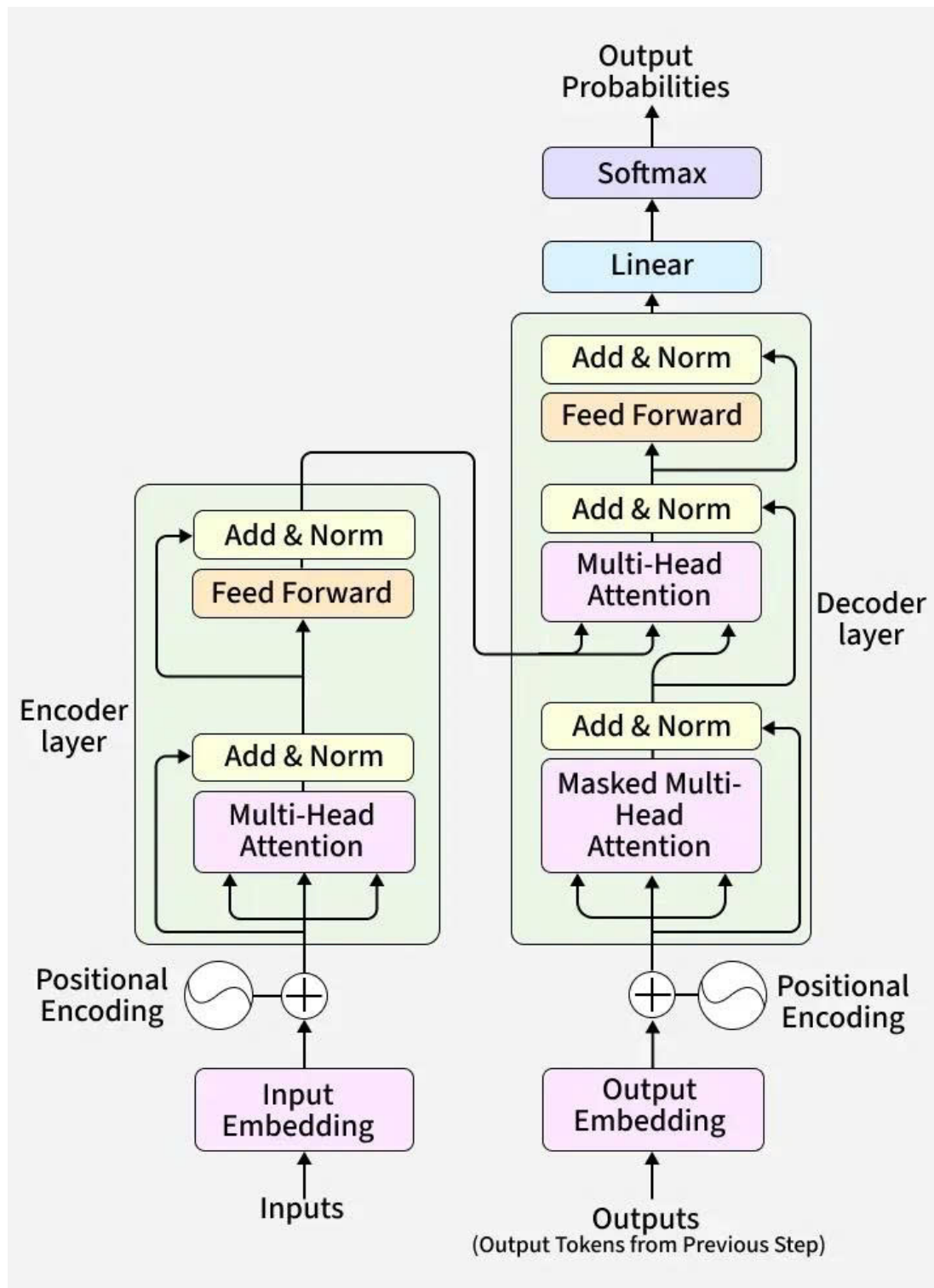
$$\delta_i(y) = P_{i,y} + \max_{y' \in \mathcal{Y}} [\delta_{i-1}(y') + A_{y',y}], \quad i = 2, 3, \dots, n, \quad (2.13)$$

Vai trò của CRF

Trong khuôn khổ đồ án, tầng CRF được đặt ngay trên đầu ra của các mạng nhằm chuyển vector ẩn tại mỗi từ thành phân phối xác suất trên toàn bộ chuỗi nhãn [26, 16, 27].

2.3.4 Mô hình Transformer

Transformer là kiến trúc mạng nơ-ron được Vaswani và cộng sự đề xuất năm 2017 trong công trình “*Attention Is All You Need*” [31], đánh dấu bước ngoặt trong xử lý ngôn ngữ tự nhiên khi loại bỏ hoàn toàn các phép hồi quy (recurrence) và tích chập (convolution), thay vào đó chỉ dựa trên cơ chế *tự chú ý* (self-attention) [32]. Nhờ khả năng tính toán song song trên toàn bộ chuỗi và mô hình hóa các phụ thuộc tầm xa một cách trực tiếp, Transformer đã trở thành nền tảng cho hầu hết các mô hình ngôn ngữ tiền huấn luyện hiện đại như BERT [33], GPT [34], RoBERTa [35] và PhoBERT [36].



Hình 2.5: Kiến trúc Transformer. [5]

2.3.4.1 Kiến trúc Encoder–Decoder

Transformer nguyên bản được thiết kế cho bài toán dịch máy theo dạng *sequence-to-sequence* [37] và bao gồm hai khối lớn (Hình ??):

- **Encoder**: gồm N lớp giống nhau xếp chồng (thường $N = 6$), nhận chuỗi đầu vào

$\mathbf{x} = (x_1, x_2, \dots, x_n)$ và biến đổi thành một dãy biểu diễn liên tục $\mathbf{Z} = (\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n)$ mã hóa toàn bộ ngữ cảnh hai chiều của câu. Mỗi lớp Encoder chứa hai khối con: (i) Multi-Head Self-Attention và (ii) Position-wise Feed-Forward Network.

- **Decoder:** cũng gồm N lớp, sinh chuỗi đầu ra $\mathbf{y} = (y_1, y_2, \dots, y_m)$ một cách *tự hồi quy* (autoregressive) — từng token một, dựa trên các token đã sinh trước đó và biểu diễn $\mathbf{Z}$ từ Encoder. Mỗi lớp Decoder có ba khối con: (i) Masked Multi-Head Self-Attention, (ii) Encoder–Decoder (Cross) Attention, và (iii) Feed-Forward Network.

Trong các mô hình ngôn ngữ tiền huấn luyện sau này, hai khối có thể được tách rời: BERT [33] và PhoBERT [36] chỉ dùng *Encoder* (encoder-only) cho các tác vụ hiểu ngôn ngữ như phân loại và NER, trong khi GPT [34] chỉ dùng *Decoder* (decoder-only) cho tác vụ sinh văn bản.

2.3.4.2 Embeddings và Mã hóa vị trí

Token Embedding.

Mỗi token đầu vào (sau khi đi qua bộ tách từ con như BPE [38] hoặc WordPiece [39]) được ánh xạ thành một vector dày đặc $\mathbf{e}_i \in \mathbb{R}^{d_{\text{model}}}$ thông qua ma trận embedding học được $\mathbf{E} \in \mathbb{R}^{|V| \times d_{\text{model}}}$, với $|V|$ là kích thước từ điển và d_{model} thường bằng 512 trong bản gốc hoặc 768 trong BERT/PhoBERT-base. Trong công trình của Vaswani và cộng sự [31], embedding đầu vào, embedding đầu ra và ma trận chiếu trước softmax đều *chia sẻ trọng số* [40], và các vector embedding được nhân thêm hệ số $\sqrt{d_{\text{model}}}$ trước khi cộng với positional encoding để cân bằng độ lớn.

Positional Encoding (PE).

Do Transformer xử lý toàn bộ chuỗi song song và không có cơ chế hồi quy như RNN/LSTM, mô hình *không có thông tin về thứ tự từ*. Để khắc phục, một vector PE $\mathbf{p}_i \in \mathbb{R}^{d_{\text{model}}}$ được cộng vào embedding tại đầu vào của cả Encoder và Decoder:

$$\mathbf{z}_i^{(0)} = \mathbf{e}_i \sqrt{d_{\text{model}}} + \mathbf{p}_i. \quad (2.14)$$

Công trình gốc [31] sử dụng *sinusoidal positional encoding*:

$$\text{PE}_{(\text{pos}, 2k)} = \sin\left(\frac{\text{pos}}{10000^{2k/d_{\text{model}}}}\right), \quad (2.15)$$

$$\text{PE}_{(\text{pos}, 2k+1)} = \cos\left(\frac{\text{pos}}{10000^{2k/d_{\text{model}}}}\right), \quad (2.16)$$

với pos là vị trí của token và k là chỉ số chiều. Cách mã hóa này có hai ưu điểm: (i) mỗi vị trí có biểu diễn duy nhất, (ii) khoảng cách tương đối giữa pos và $pos + \Delta$ có thể biểu diễn như một hàm tuyến tính của PE_{pos} , giúp tổng quát hóa tốt với chuỗi dài hơn khi huấn luyện. Các biến thể sau này bao gồm *learned positional embeddings* (dùng trong BERT [33] và PhoBERT [36]), *relative positional encoding* [41], và *Rotary Positional Embedding* (RoPE) [42].

2.3.4.3 Cơ chế Self-Attention

Self-attention (tự chú ý) là thành phần cốt lõi của Transformer [31], cho phép mỗi token trong chuỗi “nhìn” đồng thời tới mọi token khác (kể cả chính nó) để tính ra biểu diễn ngữ cảnh của mình. Từ ma trận đầu vào $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$, ba ma trận *Query*, *Key*, *Value* được tính bằng các phép chiếu tuyến tính học được:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V, \quad (2.17)$$

với $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ và $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

a) Scaled Dot-Product Attention.

Trọng số chú ý giữa các token được tính bằng tích vô hướng có chia tỉ lệ, sau đó chuẩn hóa qua softmax:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.18)$$

Hệ số $\sqrt{d_k}$ giúp ổn định gradient khi d_k lớn, tránh việc softmax rơi vào vùng bão hòa [31]. Khi tính cho Decoder, một *mask* tam giác được áp vào ma trận điểm số trước softmax (đặt $-\infty$ cho các vị trí tương lai) để đảm bảo tính tự hồi quy.

2.3.4.4 Multi-Head Attention

Thay vì thực hiện một phép attention duy nhất với d_{model} chiều, Transformer chia không gian thành h “đầu” song song (thường $h = 8$ với $d_k = d_v = d_{\text{model}}/h = 64$), mỗi đầu học một phép chiếu tuyến tính riêng:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q, \mathbf{X}\mathbf{W}_i^K, \mathbf{X}\mathbf{W}_i^V), \quad (2.19)$$

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (2.20)$$

với $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Multi-Head Attention mang lại hai lợi ích quan trọng: (i) cho phép mô hình *đồng thời tập trung vào nhiều khía cạnh ngữ nghĩa và cú pháp khác nhau* (ví dụ một đầu học quan

hệ chủ-vị, đầu khác học quan hệ đại từ-danh từ); (ii) giữ tổng chi phí tính toán tương đương với single-head attention nhờ chia nhỏ chiều ẩn [31].

2.3.4.5 Position-wise Feed-Forward Network

Sau khối chú ý, mỗi vị trí trong chuỗi được xử lý độc lập bởi một mạng truyền thẳng hai lớp *giống nhau cho mọi vị trí* (do đó gọi là “position-wise”):

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2, \quad (2.21)$$

với $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ và $d_{\text{ff}} = 2048$ trong bản gốc [31]. Mặc dù trọng số được chia sẻ giữa các vị trí trong cùng một lớp, mỗi lớp Encoder/Decoder có một bộ tham số FFN riêng. Khối này có thể xem như một tích chập 1D với kernel size 1, đóng vai trò *biến đổi phi tuyến* bổ sung dung lượng biểu diễn cho mô hình. Các biến thể sau này thay ReLU bằng GELU [43] (như trong BERT, PhoBERT) để cải thiện hiệu năng.

2.3.4.6 Add & Norm: Kết nối tắt và Chuẩn hóa lớp

Mỗi khối con (Self-Attention, Cross-Attention, FFN) đều được bao bọc bởi một *kết nối tắt* (residual connection) [44] và sau đó là *Layer Normalization* [45]:

$$\text{Output} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})). \quad (2.22)$$

Vai trò của hai thành phần:

- **Residual Connection** ($\mathbf{x} + \text{Sublayer}(\mathbf{x})$): tạo “đường tắt” cho gradient lan truyền ngược trực tiếp qua các lớp sâu, giảm hiện tượng *vanishing gradient* và cho phép huấn luyện mạng rất sâu (hàng chục đến hàng trăm lớp như trong GPT-3 [46], PhoBERT-large [36]).
- **Layer Normalization**: chuẩn hóa kích hoạt theo từng vị trí (trên trục đặc trưng), giúp ổn định phân phối đầu vào của lớp tiếp theo và tăng tốc hội tụ [45]. Khác với Batch Normalization [47], LayerNorm không phụ thuộc vào kích thước batch nên phù hợp với dữ liệu chuỗi có độ dài thay đổi.

Lưu ý rằng bản gốc [31] áp dụng LayerNorm *sau* khối con (*Post-LN*), trong khi các biến thể hiện đại như GPT-2 [48] và PhoBERT [36] thường dùng *Pre-LN* [49] — áp dụng LayerNorm *trước* khối con — giúp huấn luyện ổn định hơn và không cần kỹ thuật *learning rate warm-up* phức tạp.

2.3.5 Mô hình PhoBERT

Kiến trúc PhoBERT

PhoBERT [36] là một mô hình ngôn ngữ tiền huấn luyện đơn ngữ dành riêng cho tiếng Việt, "Pho" trong PhoBERT chính là tên một món ăn nổi tiếng của Việt Nam "Phở". Mô hình được đề xuất bởi Nguyễn Quốc Đạt và Nguyễn Anh Tuấn, được xây dựng dựa trên kiến trúc của RoBERTa [35]. Trong nghiên cứu này, mô hình PhoBERT-base-v2 được sử dụng làm backbone cho bài toán nhận dạng thực thể có tên. Mô hình bao gồm 12 tầng Transformer Encoder, 12 attention heads, kích thước biểu diễn ẩn 768 chiều, với tổng cộng khoảng 135 triệu tham số. Độ dài đầu vào tối đa được hỗ trợ là 256 token.

Dữ liệu tiền huấn luyện

Phiên bản PhoBERT-base-v2 được huấn luyện trên tập dữ liệu tiếng Việt quy mô lớn gồm khoảng 140 GB văn bản:

- khoảng 20 GB dữ liệu từ Wikipedia tiếng Việt và các bài báo tin tức được làm sạch.
- Khoảng 120 GB dữ liệu tiếng Việt thu thập từ tập dữ liệu OSCAR phiên bản OSCAR-2301

Trước khi huấn luyện, dữ liệu được tách từ bằng công cụ RDRSegmenter thuộc bộ công cụ VnCoreNLP. Sau đó, phương pháp BPE được áp dụng với từ vựng gồm khoảng 64.000 subword token nhằm xử lý hiệu quả đặc điểm từ ghép trong tiếng Việt.

PhoBERT [36] sử dụng chiến lược tiền huấn luyện Masked Language Modeling (MLM) tương tự RoBERTa [35]. Trong quá trình huấn luyện, một phần token trong câu được che khuất và mô hình học cách dự đoán các token bị ẩn dựa trên ngữ cảnh hai chiều. Việc loại bỏ nhiệm vụ Next Sentence Prediction (NSP) của BERT và tối ưu hóa các siêu tham số huấn luyện giúp PhoBERT [36] đạt hiệu năng vượt trội trên nhiều tác vụ xử lý ngôn ngữ tiếng Việt.

2.4 Các kỹ thuật tối ưu hóa mô hình

Trong bối cảnh triển khai các mô hình học sâu, đặc biệt là các Mô hình Ngôn ngữ Lớn (LLMs) như Transformer hay PhoBERT lên các môi trường hạn chế về tài nguyên (thiết bị biên, máy chủ cấu hình thấp), việc tối ưu hóa là yêu cầu bắt buộc. Đồ án tập trung nghiên cứu và tích hợp bốn kỹ thuật tối ưu cốt lõi nhằm giảm thiểu chi phí phần cứng mà vẫn bảo toàn tối đa độ chính xác của hệ thống nhận diện thực thể.

2.4.1 Low-Rank Adaptation (LoRA)

Trong kỷ nguyên của các mô hình học sâu quy mô lớn, việc tinh chỉnh (Fine-tuning) toàn bộ tham số (Full Fine-tuning) của các mô hình ngôn ngữ tiền huấn luyện (Pre-trained Language Models - PLMs) như BERT, GPT hay PhoBERT đã trở thành phương pháp tiêu chuẩn để đạt được hiệu năng tối ưu trên các tác vụ chuyên biệt. Tuy nhiên, phương pháp tiếp cận này đang đối mặt với những thách thức nghiêm trọng về mặt kỹ thuật và hạ tầng phần cứng:

1. **Bùng nổ chi phí lưu trữ:** Việc lưu trữ toàn bộ các bản sao tham số của mô hình cho từng tác vụ riêng biệt là cực kỳ lãng phí bộ nhớ dữ liệu. Với một mô hình nền tảng, việc tạo ra hàng chục phiên bản fine-tune cho hàng chục tác vụ khác nhau sẽ làm tăng chi phí lưu trữ lên theo cấp số nhân.
2. **Yêu cầu VRAM khổng lồ khi huấn luyện:** Trong quá trình lan truyền ngược (Backward Pass), bộ tối ưu hóa (Optimizer) như AdamW yêu cầu lưu trữ không chỉ các trọng số (Weights), mà còn cả Gradient và các trạng thái nội tại (Optimizer States) bao gồm khoảng khắc bậc một (Momentum) và bậc hai (Variance). Điều này đẩy mức tiêu thụ bộ nhớ đồ họa (VRAM) lên gấp 3 đến 4 lần dung lượng gốc của mô hình, vượt quá khả năng chi trả của các phòng thí nghiệm hoặc doanh nghiệp vừa và nhỏ.
3. **Hiện tượng Catastrophic Forgetting:** Khi ép buộc một mô hình đã học được các đặc trưng ngôn ngữ tổng quát phải cập nhật toàn bộ trọng số trên một tập dữ liệu chuyên biệt quy mô nhỏ (như dữ liệu y tế), mô hình có xu hướng phá vỡ cấu trúc không gian biểu diễn cũ, dẫn đến việc mất đi năng lực hiểu ngôn ngữ tự nhiên tổng quát.

Để giải quyết triệt để các hạn chế trên, Edward Hu và các cộng sự tại Microsoft (2021) đã đề xuất kỹ thuật **Low-Rank Adaptation (LoRA)**. Đây là một trong những phương pháp tinh chỉnh hiệu quả tham số (Parameter-Efficient Fine-Tuning - PEFT) tiên tiến nhất hiện nay, hoạt động dựa trên giả thuyết toán học về "Hạng nội tại thấp" (Low Intrinsic Dimension) của không gian học tập trong mạng thần kinh.

2.4.1.1 Cơ sở lý thuyết và Bản chất Toán học của Phân rã Hạng thấp

Ý tưởng cốt lõi của LoRA bắt nguồn từ các nghiên cứu trước đó về bản chất của quá trình quá tham số hóa (Over-parameterization) trong mạng học sâu. Các nghiên cứu này chỉ ra rằng, mặc dù các ma trận trọng số trong mô hình lớn có số chiều không gian rất cao, xung lực thay đổi của trọng số (ΔW) trong quá trình thích ứng với tác vụ mới thực chất chỉ nằm trong một không gian con có hạng nội tại (Intrinsic Rank) rất thấp.

Dựa trên nguyên lý này, LoRA đóng băng hoàn toàn ma trận trọng số tiền huấn luyện gốc $W_0 \in \mathbb{R}^{d \times k}$ và xấp xỉ ma trận cập nhật trọng số $\Delta W \in \mathbb{R}^{d \times k}$ thông qua việc tích hợp cấu trúc phân rã tích của hai ma trận hạng thấp (Low-rank decomposition matrices) là $B \in \mathbb{R}^{d \times r}$ và $A \in \mathbb{R}^{r \times k}$, với điều kiện hạng $r \ll \min(d, k)$.

$$W = W_0 + \Delta W = W_0 + B \cdot A \quad (2.23)$$

Mô tả toán học của quá trình lan truyền xuôi (Forward Pass) khi áp dụng toán tử LoRA vào một véc-tơ đầu vào $x \in \mathbb{R}^d$ được thể hiện qua phương trình:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x \quad (2.24)$$

Trong đó:

- W_0 : Ma trận trọng số gốc của mô hình tiền huấn luyện (được đóng băng hoàn toàn, không tính toán gradient và không cập nhật).
- B, A : Các ma trận LoRA chịu trách nhiệm học các đặc trưng của tác vụ mới (được tính toán gradient và cập nhật liên tục).
- r : Siêu tham số quy định hạng (Rank) của không gian con. Giá trị của r thông thường được chọn rất nhỏ ($r \in \{4, 8, 16, 32, 64\}$) so với kích thước thực tế của d hay k (thường lên đến hàng nghìn đơn vị).
- α : Hệ số tỷ lệ cố định (Scaling hyperparameter). Đại lượng $\frac{\alpha}{r}$ đóng vai trò như một hằng số chuẩn hóa biên độ đặc trưng. Khi tinh chỉnh hoặc thay đổi giá trị của hạng r , việc giữ nguyên α giúp người huấn luyện không cần phải thiết lập lại tốc độ học (Learning Rate) của bộ tối ưu hóa, tăng tính ổn định cho chu trình hội tụ.

2.4.1.2 Chiến lược khởi tạo ma trận (Initialization Strategy)

Để đảm bảo quá trình huấn luyện diễn ra một cách mượt mà và không gây ra nhiễu loạn ở những bước lặp đầu tiên, LoRA áp dụng một chiến lược khởi tạo ma trận bất đối xứng rất đặc trưng:

- **Ma trận A** : Được khởi tạo ngẫu nhiên theo phân phối Gaussian (hoặc phân phối đều) với kỳ vọng bằng 0 và độ lệch chuẩn nhỏ: $A \sim \mathcal{N}(0, \sigma^2)$.
- **Ma trận B** : Được khởi tạo bằng một ma trận không tuyệt đối: $B = 0$.

Hệ quả toán học của chiến lược khởi tạo này là tại thời điểm bắt đầu huấn luyện ($t = 0$), tích của hai ma trận hoàn toàn bằng không:

$$\Delta W = B \cdot A = 0 \cdot A = 0 \quad (2.25)$$

Do $\Delta W = 0$, kết quả đầu ra đầu tiên của mạng thần kinh áp dụng LoRA hoàn toàn trùng khớp với kết quả đầu ra của mô hình tiền huấn luyện gốc ($h = W_0x$). Điều này giúp loại bỏ hoàn toàn rủi ro sai số khởi đầu, bảo toàn nguyên vẹn năng lực tri thức nền tảng của mô hình trước khi nó bắt đầu tiếp thu các dữ liệu chuyên biệt.

2.4.1.3 Vị trí tích hợp LoRA trong Kiến trúc Transformer

Trong một kiến trúc Transformer tiêu chuẩn (như BERT hoặc PhoBERT), các lớp tuyến tính xuất hiện ở cả khối chú ý đa đầu (Multi-Head Attention) và khối mạng lan truyền thẳng (Feed-Forward Network - FFN). Về mặt lý thuyết, LoRA có thể được áp dụng vào bất kỳ ma trận trọng số của lớp tuyến tính nào. Tuy nhiên, để tối ưu hóa hiệu năng, cấu trúc LoRA thường được ưu tiên tích hợp vào các ma trận chiều của cơ chế Self-Attention:

1. **Ma trận Trọng số Truy vấn và Khóa (W_q, W_k):** Tích hợp LoRA vào đây giúp mô hình điều chỉnh lại cách thức tính toán ma trận điểm số chú ý (Attention Scores), thay đổi trọng tâm liên kết giữa các từ ngữ trong ngữ cảnh mới.
2. **Ma trận Trọng số Giá trị (W_v):** Đây là vị trí quan trọng nhất. Ma trận W_v chịu trách nhiệm trực tiếp đại diện cho nội dung thông tin được trích xuất sau cơ chế chú ý. Thực nghiệm cho thấy việc chèn LoRA vào W_v mang lại hiệu quả cải thiện độ chính xác cao nhất trên các tác vụ downstream.
3. **Ma trận Đầu ra và Khối FFN (W_o, W_{in}, W_{out}):** Việc mở rộng LoRA sang các lớp FFN (gồm hai lớp tuyến tính phân tách bởi hàm kích hoạt phi tuyến GeLU) giúp mô hình tăng cường năng lực lưu trữ tri thức tính mang tính chất sự kiện của miền dữ liệu mới.

Để chứng minh khả năng tiết kiệm tham số đáng kể của LoRA, xét một lớp tuyến tính trong cấu trúc PhoBERT-base có kích thước chiều ẩn $d = k = 768$.

- Số lượng tham số của ma trận gốc W_0 : $768 \times 768 = 589,824$ tham số.
- Nếu áp dụng LoRA với hạng $r = 8$: Số tham số của ma trận A là $8 \times 768 = 6,144$; số tham số của ma trận B là $768 \times 8 = 6,144$. Tổng số tham số cần huấn luyện chỉ là: $6,144 + 6,144 = 12,288$ tham số.

Như vậy, LoRA đã giảm số lượng tham số cần tính toán gradient từ gần 590,000 xuống chỉ còn hơn 12,000 (tiết kiệm tới hơn 97.9% số lượng tham số trên một lớp).

2.4.1.4 Cơ chế Hợp nhất Trọng số (Weight Merging) và Suy luận Không độ trễ

Một trong những ưu điểm vượt trội giúp LoRA đánh bại hoàn toàn các phương pháp PEFT truyền thống khác (như Adapter layers) chính là khả năng loại bỏ hoàn toàn độ trễ suy luận (Zero Inference Latency) khi triển khai vào thực tế sản xuất.

Đối với các phương pháp chèn thêm các khối chức năng (Adapter) theo dạng tuần lặ (Serial) vào giữa các tầng của Transformer, khi mô hình thực hiện suy luận, luồng dữ liệu bắt buộc phải di chuyển tuần tự qua các lớp bổ sung này. Điều này làm gia tăng độ sâu đồ thị tính toán của mô hình, trực tiếp tạo ra một "nút thắt cổ chai" về tốc độ xử lý thời gian thực.

LoRA giải quyết bài toán này một cách triệt để nhờ vào tính chất tuyến tính của phép toán nhân ma trận. Sau khi quá trình huấn luyện kết thúc và mô hình đã đạt được độ hội tụ tối ưu, ta có thể thực hiện một phép toán cộng ma trận tĩnh để hợp nhất (merge) các trọng số đã học được từ ma trận hạng thấp vào trực tiếp ma trận trọng số gốc ban đầu:

$$W_{merged} = W_0 + \frac{\alpha}{r}(B \cdot A) \quad (2.26)$$

Do $W_{merged} \in \mathbb{R}^{d \times k}$, cấu trúc và kích thước của ma trận sau khi hợp nhất hoàn toàn trùng khớp với định dạng của mô hình nguyên bản. Khi triển khai mô hình này lên các hệ thống chạy thực tế, phần cứng chỉ cần thực hiện một phép toán tuyến tính duy nhất ($h = W_{merged}x$). Do đó, hệ thống hoàn toàn không phải gánh chịu bất kỳ một chi phí tính toán phát sinh nào, đảm bảo tốc độ suy luận đạt mức tối đa tương đương với mô hình Full Fine-tuning gốc.

Hơn thế nữa, cơ chế này cho phép chuyển đổi cực kỳ linh hoạt giữa các tác vụ. Nếu một máy chủ cần phục vụ nhiều tác vụ NER khác nhau (ví dụ: một phiên bản NER cho bệnh án Covid-19, một phiên bản cho bệnh án tim mạch), hệ thống chỉ cần lưu trữ một bản sao duy nhất của mô hình gốc W_0 dưới dạng bộ nhớ chia sẻ (Shared Memory) và lưu trữ các cặp ma trận LoRA (A, B) siêu nhẹ cho từng tác vụ. Khi cần chuyển đổi tác vụ, hệ thống chỉ cần thực hiện phép trừ ma trận để loại bỏ cấu hình cũ và cộng ma trận để nạp cấu hình mới trong thời gian phần mili-giây:

$$W_{task2} = W_{merged_task1} - \frac{\alpha_1}{r_1}(B_1A_1) + \frac{\alpha_2}{r_2}(B_2A_2) \quad (2.27)$$

2.4.1.5 Phân tích So sánh Giữa LoRA và Các Phương pháp PEFT Khác

Để làm rõ vị thế của LoRA, bảng dưới đây thực hiện phân tích so sánh định lượng và định tính dựa trên các tiêu chí cốt lõi trong vận hành hệ thống học sâu:

Bảng 2.1: Bảng so sánh hiệu năng giữa LoRA và các phương pháp tinh chỉnh phổ biến

Phương pháp	Tham số huấn luyện	Độ trễ suy luận	Tiết kiệm VRAM	Giữ chiều dài chuỗi
Full Fine-Tuning	100%	Không tăng	Thấp nhất	Có
Houlsby Adapter	1%-3%	Có tăng (Tuần tự)	Khá	Có
Prefix Tuning	1%-2%	Không tăng	Tốt	Không (tốn không gian đệm)
LoRA	< 1%	Không tăng (Merged)	Rất cao	Có

2.4.1.6 Ý nghĩa của LoRA đối với bài toán Nhận diện thực thể có tên (PhoNER_COVID19)

Trong khuôn khổ nghiên cứu của đề án, việc lựa chọn LoRA làm giải pháp tối ưu hóa cấu trúc cho mô hình nền tảng PhoBERT mang lại những giá trị thực tiễn mang tính quyết định:

- Khắc phục giới hạn kích thước tập dữ liệu:** Tập dữ liệu PhoNER_COVID19 mặc dù được xây dựng công phu nhưng quy mô vẫn ở mức giới hạn so với hàng tỷ từ vựng mà PhoBERT được học trong giai đoạn tiền huấn luyện. Nếu áp dụng Full Fine-tuning, mô hình rất dễ rơi vào trạng thái quá khớp (Overfitting), chỉ ghi nhớ máy móc các câu trong tập Train mà mất đi khả năng suy luận tổng quát trên các văn bản dịch tễ thực tế ngoài xã hội. LoRA đóng vai trò như một bộ lọc điều hòa (Regularization), giới hạn không gian cập nhật tham số, giúp mô hình tập trung tối đa vào việc liên kết các thực thể đặc thù (SYMPTOM, LOCATION, PATIENT_ID) mà không làm méo mó cấu trúc ngữ pháp tiếng Việt gốc.
- Tối ưu hóa tài nguyên nghiên cứu:** Với cấu hình phần cứng giới hạn trong môi trường học đường, kỹ thuật LoRA cho phép tăng kích thước phân đoạn dữ liệu (Batch Size) lên gấp nhiều lần trong quá trình huấn luyện nhờ giải phóng được phần lớn bộ nhớ VRAM dành cho Optimizer States. Chu trình thử nghiệm (Experimentation Loop) từ đó được tăng tốc, cho phép người nghiên cứu thực hiện dò tìm siêu tham số trên nhiều cấu hình mạng một cách nhanh chóng.

Tóm lại, Low-Rank Adaptation không chỉ đơn thuần là một giải pháp nén hay tiết kiệm bộ nhớ, mà nó đại diện cho một tư duy thiết kế hệ thống thông minh: cô lập tri thức nền tảng và chỉ dẫn dắt sự thay đổi thông qua các không gian hình học hạng thấp. Sự tích hợp của LoRA vào hệ thống nhận diện thực thể tiếng Việt tạo ra một bước đi vững chắc, giải quyết đồng thời cả bài toán về độ chính xác học thuật và bài toán tối ưu hạ tầng triển khai.

2.4.2 Knowledge Distillation (Chưng cất tri thức)

Trong xu thế phát triển của ngành Học sâu (Deep Learning) và Xử lý ngôn ngữ tự nhiên (NLP), các mô hình ngôn ngữ tiền huấn luyện quy mô lớn (Pre-trained Language Models) như BERT, RoBERTa hay PhoBERT đã thiết lập những cột mốc hiệu năng vượt trội trên hầu hết các bài toán thực tế. Tuy nhiên, rào cản lớn nhất của các mô hình này khi triển khai vào môi trường sản xuất (production) hoặc các thiết bị biên (edge devices) chính là kích thước tham số khổng lồ, chi phí tính toán đắt đỏ và độ trễ suy luận (inference latency) cao.

Để giải quyết mâu thuẫn cốt lõi giữa độ chính xác và hiệu năng vận hành, kỹ thuật **Knowledge Distillation (KD - Chưng cất tri thức)** do Geoffrey Hinton và các cộng sự đề xuất vào năm 2015 đã trở thành một trong những phương pháp tiếp cận hiệu quả nhất. Kỹ thuật này cho phép chuyển giao một cách hệ thống năng lực biểu diễn và tri thức từ một mạng thần kinh lớn, phức tạp (được gọi là mô hình Giáo viên - Teacher Model) sang một mạng thần kinh nhỏ gọn, tối ưu hơn về mặt cấu trúc (được gọi là mô hình Học sinh - Student Model) mà không làm suy giảm nghiêm trọng độ chính xác của hệ thống.

2.4.2.1 Cơ sở toán học và cơ chế chuyển giao "Tri thức tối" (Dark Knowledge)

Ý tưởng nền tảng của Chưng cất tri thức xuất phát từ nhận thức rằng: Tri thức của một mô hình học sâu không chỉ nằm ở nhãn dự đoán cuối cùng (Hard Labels), mà còn được phân bố trong toàn bộ phân phối xác suất đầu ra giữa các lớp (Soft Labels). Phân phối xác suất này chứa đựng các thông tin ẩn sâu về cấu trúc liên kết nội tại giữa các thực thể, được gọi là "Tri thức tối" (Dark Knowledge).

Ví dụ, trong bài toán Nhận diện thực thể y tế (NER), đối với một token cụ thể, nhãn gốc (Ground Truth) có thể là B-SYMPTOM (Triệu chứng). Tuy nhiên, mô hình Giáo viên có thể đưa ra phân phối xác suất: 70% cho B-SYMPTOM, 25% cho B-DISEASE (Bệnh lý), và chỉ 5% cho O (Nằm ngoài thực thể). Con số 25% dành cho B-DISEASE phản ánh một tri thức quan trọng mà mô hình Giáo viên đã học được: mối tương quan ngữ nghĩa rất gần giữa "Triệu chứng" và "Bệnh lý" trong văn cảnh y khoa. Nếu chỉ sử dụng nhãn cứng một-hot (One-hot Vector) để huấn luyện mô hình Học sinh từ đầu, toàn bộ thông tin về mối tương quan này sẽ bị triệt tiêu hoàn toàn.

Để trích xuất và khuếch đại lượng tri thức tối này, hàm kích hoạt Softmax truyền thống được điều chỉnh bằng cách tích hợp thêm tham số Nhiệt độ (T - Temperature), được định nghĩa bởi công thức:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2.28)$$

Trong đó:

- z_i : Là giá trị Logit (đầu ra chưa qua kích hoạt) của lớp thứ i .
- T : Là tham số nhiệt độ (một số thực dương). Khi $T = 1$, phương trình trở về hàm Softmax tiêu chuẩn. Khi $T > 1$, phân phối xác suất đầu ra sẽ trở nên "mềm" hơn (smoother), làm nổi bật xác suất của các lớp không phải là nhãn chính, giúp mô hình Học sinh dễ dàng tiếp thu cấu trúc không gian nhãn.

2.4.2.2 Cấu trúc Hàm mất mát tổng hợp (Combined Loss Function)

Quá trình huấn luyện mô hình Học sinh dưới sự dẫn dắt của mô hình Giáo viên được điều phối bởi một hàm mất mát đa mục tiêu (Multi-task Loss Function). Hàm mất mát này là sự kết hợp tuyến tính giữa hai thành phần: Hàm mất mát giám sát truyền thống (Student Loss) và Hàm mất mát chưng cất (Distillation Loss).

Sơ đồ toán học tổng quát của quá trình chưng cất tri thức được mô tả qua phương trình sau:

$$L_{total} = \alpha L_{CE}(y, P_S) + (1 - \alpha) T^2 \cdot D_{KL}(P_T^T \parallel P_S^T) \quad (2.29)$$

Trong đó:

- $L_{CE}(y, P_S)$: Là hàm mất mát Cross-Entropy tiêu chuẩn tính toán giữa nhãn cứng thực tế y (Ground Truth) và phân phối xác suất đầu ra của mô hình Học sinh P_S tại nhiệt độ $T = 1$.
- $D_{KL}(P_T^T \parallel P_S^T)$: Là độ phân kỳ Kullback-Leibler (KL Divergence), đo lường sự khác biệt giữa phân phối xác suất mềm của Giáo viên (P_T^T) và Học sinh (P_S^T) được tính toán tại cùng một mức nhiệt độ $T > 1$. Công thức chi tiết của D_{KL} được biểu diễn là:

$$D_{KL}(P_T^T \parallel P_S^T) = \sum_i p_i^T \log \left(\frac{p_i^T}{s_i^T} \right) \quad (2.30)$$

Với p_i^T và s_i^T lần lượt là xác suất mềm lớp i của Giáo viên và Học sinh.

- α : Phép cân bằng trọng số ($0 \leq \alpha \leq 1$) điều phối mức độ đóng góp của hai nguồn tri thức. Trong thực nghiệm, hệ số T^2 được nhân vào thành phần D_{KL} nhằm mục đích cân bằng quy mô gradient (gradient scale), do khi tính đạo hàm của hàm Softmax điều chỉnh theo nhiệt độ, biên độ gradient sẽ bị giảm đi một lượng tỷ lệ thuận với $1/T^2$.

2.4.3 Knowledge Distillation (Chưng cất tri thức)

Trong xu thế phát triển của ngành Học sâu (Deep Learning) và Xử lý ngôn ngữ tự nhiên (NLP), các mô hình ngôn ngữ tiền huấn luyện quy mô lớn (Pre-trained Language Models)

như BERT, RoBERTa hay PhoBERT đã thiết lập những cột mốc hiệu năng vượt trội trên hầu hết các bài toán thực tế. Tuy nhiên, rào cản lớn nhất của các mô hình này khi triển khai vào môi trường sản xuất (production) hoặc các thiết bị biên (edge devices) chính là kích thước tham số khổng lồ, chi phí tính toán đắt đỏ và độ trễ suy luận (inference latency) cao.

Để giải quyết mâu thuẫn cốt lõi giữa độ chính xác và hiệu năng vận hành, kỹ thuật **Knowledge Distillation (KD - Chưng cất tri thức)** do Geoffrey Hinton và các cộng sự đề xuất vào năm 2015 đã trở thành một trong những phương pháp tiếp cận hiệu quả nhất. Kỹ thuật này cho phép chuyển giao một cách hệ thống năng lực biểu diễn và tri thức từ một mạng thần kinh lớn, phức tạp (được gọi là mô hình Giáo viên - Teacher Model) sang một mạng thần kinh nhỏ gọn, tối ưu hơn về mặt cấu trúc (được gọi là mô hình Học sinh - Student Model) mà không làm suy giảm nghiêm trọng độ chính xác của hệ thống.

2.4.3.1 Cơ sở toán học và cơ chế chuyển giao "Tri thức tối" (Dark Knowledge)

Ý tưởng nền tảng của Chưng cất tri thức xuất phát từ nhận thức rằng: Tri thức của một mô hình học sâu không chỉ nằm ở nhãn dự đoán cuối cùng (Hard Labels), mà còn được phân bổ trong toàn bộ phân phối xác suất đầu ra giữa các lớp (Soft Labels). Phân phối xác suất này chứa đựng các thông tin ẩn sâu về cấu trúc liên kết nội tại giữa các thực thể, được gọi là "Tri thức tối" (Dark Knowledge).

Ví dụ, trong bài toán Nhận diện thực thể y tế (NER), đối với một token cụ thể, nhãn gốc (Ground Truth) có thể là B-SYMPTOM (Triệu chứng). Tuy nhiên, mô hình Giáo viên có thể đưa ra phân phối xác suất: 70% cho B-SYMPTOM, 25% cho B-DISEASE (Bệnh lý), và chỉ 5% cho O (Nằm ngoài thực thể). Con số 25% dành cho B-DISEASE phản ánh một tri thức quan trọng mà mô hình Giáo viên đã học được: mối tương quan ngữ nghĩa rất gần giữa "Triệu chứng" và "Bệnh lý" trong văn cảnh y khoa. Nếu chỉ sử dụng nhãn cứng một-hot (One-hot Vector) để huấn luyện mô hình Học sinh từ đầu, toàn bộ thông tin về mối tương quan này sẽ bị triệt tiêu hoàn toàn.

Để trích xuất và khuếch đại lượng tri thức tối này, hàm kích hoạt Softmax truyền thống được điều chỉnh bằng cách tích hợp thêm tham số Nhiệt độ (T - Temperature), được định nghĩa bởi công thức:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2.31)$$

Trong đó:

- z_i : Là giá trị Logit (đầu ra chưa qua kích hoạt) của lớp thứ i .

- T : Là tham số nhiệt độ (một số thực dương). Khi $T = 1$, phương trình trở về hàm Softmax tiêu chuẩn. Khi $T > 1$, phân phối xác suất đầu ra sẽ trở nên "mềm" hơn (smoother), làm nổi bật xác suất của các lớp không phải là nhãn chính, giúp mô hình Học sinh dễ dàng tiếp thu cấu trúc không gian nhãn.

2.4.3.2 Cấu trúc Hàm mất mát tổng hợp (Combined Loss Function)

Quá trình huấn luyện mô hình Học sinh dưới sự dẫn dắt của mô hình Giáo viên được điều phối bởi một hàm mất mát đa mục tiêu (Multi-task Loss Function). Hàm mất mát này là sự kết hợp tuyến tính giữa hai thành phần: Hàm mất mát giám sát truyền thống (Student Loss) và Hàm mất mát chưng cất (Distillation Loss).

Sơ đồ toán học tổng quát của quá trình chưng cất tri thức được mô tả qua phương trình sau:

$$L_{total} = \alpha L_{CE}(y, P_S) + (1 - \alpha) T^2 \cdot D_{KL}(P_T^T \parallel P_S^T) \quad (2.32)$$

Trong đó:

- $L_{CE}(y, P_S)$: Là hàm mất mát Cross-Entropy tiêu chuẩn tính toán giữa nhãn cứng thực tế y (Ground Truth) và phân phối xác suất đầu ra của mô hình Học sinh P_S tại nhiệt độ $T = 1$.
- $D_{KL}(P_T^T \parallel P_S^T)$: Là độ phân kỳ Kullback-Leibler (KL Divergence), đo lường sự khác biệt giữa phân phối xác suất mềm của Giáo viên (P_T^T) và Học sinh (P_S^T) được tính toán tại cùng một mức nhiệt độ $T > 1$. Công thức chi tiết của D_{KL} được biểu diễn là:

$$D_{KL}(P_T^T \parallel P_S^T) = \sum_i p_i^T \log \left(\frac{p_i^T}{s_i^T} \right) \quad (2.33)$$

Với p_i^T và s_i^T lần lượt là xác suất mềm lớp i của Giáo viên và Học sinh.

- α : Phép cân bằng trọng số ($0 \leq \alpha \leq 1$) điều phối mức độ đóng góp của hai nguồn tri thức. Trong thực nghiệm, hệ số T^2 được nhân vào thành phần D_{KL} nhằm mục đích cân bằng quy mô gradient (gradient scale), do khi tính đạo hàm của hàm Softmax điều chỉnh theo nhiệt độ, biên độ gradient sẽ bị giảm đi một lượng tỷ lệ thuận với $1/T^2$.

2.4.3.3 Các cơ chế điều phối học tập trong Chưng cất tri thức (Distillation Schemes)

Dựa trên mối quan hệ về mặt thời gian huấn luyện và sự đồng bộ hóa giữa mô hình Giáo viên và mô hình Học sinh, các cơ chế chưng cất tri thức được chia thành ba kịch bản vận hành cốt lõi:

a) Offline Knowledge Distillation (Chưng cất tri thức ngoại tuyến)

Đây là cơ chế truyền thống và được áp dụng phổ biến nhất trong thực tế. Quy trình này diễn ra theo hai giai đoạn độc lập: Đầu tiên, mô hình Giáo viên có kích thước lớn (như PhoBERT) được huấn luyện đầy đủ trên tập dữ liệu đích cho đến khi đạt độ hội tụ tối ưu và cố định toàn bộ trọng số (frozen weights). Sau đó, mô hình Học sinh mới bắt đầu quá trình huấn luyện bằng cách trích xuất tri thức tĩnh (như các vector Logit hoặc Hidden States) được kết xuất sẵn từ Giáo viên.

Ưu điểm: Quy trình thực hiện đơn giản, dễ kiểm soát và không yêu cầu tài nguyên phần cứng cực đoan do không phải lan truyền ngược (backpropagation) qua mô hình Giáo viên trong pha chưng cất. **Nhược điểm:** Tri thức chuyển giao là tri thức tĩnh. Nếu mô hình Giáo viên quá lớn, khoảng cách năng lực biểu diễn quá xa, mô hình Học sinh sẽ gặp hiện tượng quá tải và không thể mô phỏng chính xác hành vi của Giáo viên.

b) Online Knowledge Distillation (Chưng cất tri thức trực tuyến)

Nhằm khắc phục hạn chế của cơ chế ngoại tuyến, chưng cất trực tuyến thực hiện cập nhật song song cả mô hình Giáo viên và mô hình Học sinh trong cùng một chu kỳ huấn luyện. Tại đây, cấu trúc không bắt buộc phải tuân theo một chiều cố định (Giáo viên dẫn dắt Học sinh), mà cả hai mô hình có thể cùng cập nhật lẫn nhau, hoặc một nhóm nhiều mô hình con (Peers) cùng học hỏi từ một mạng lưới tri thức chung (Deep Mutual Learning).

Ưu điểm: Loại bỏ được bài toán "khoảng cách năng lực" nhờ việc hai mô hình cùng tiến hóa và tối ưu hóa đồng thời. Giáo viên có thể tự điều chỉnh phân phối tri thức sao cho phù hợp với tốc độ tiếp thu của Học sinh. **Nhược điểm:** Chi phí tính toán cực kỳ lớn và thuật toán đồng bộ hóa phức tạp, đòi hỏi bộ nhớ GPU đủ lớn để duy trì biểu đồ tính toán (computational graph) của cả hai mô hình cùng lúc.

c) Self-Knowledge Distillation (Tự chưng cất tri thức)

Là một biến thể đặc biệt, nơi mạng thần kinh tự đóng vai trò là Giáo viên và Học sinh của chính nó. Tri thức từ các tầng sâu hơn (deeper layers/blocks), chứa đựng nhiều thông tin ngữ nghĩa trừu tượng cấp cao, sẽ được chưng cất ngược trở lại để dẫn dắt quá trình học của các tầng nông hơn (shallow layers). Một nhánh tiếp cận khác của Tự chưng cất là kỹ thuật tăng cường dữ liệu dựa trên sự nhất quán đầu ra (Data Augmentation / Consistency Regularization) của cùng một mô hình dưới các điều kiện dropout khác nhau.

Ưu điểm: Hoàn toàn độc lập, không cần tìm kiếm hoặc huấn luyện một mô hình Giáo viên công kênh từ bên ngoài, giúp tiết kiệm đáng kể thời gian thiết kế kiến trúc hệ thống. **Nhược điểm:** Giới hạn trần hiệu năng bị phụ thuộc hoàn toàn vào dung lượng tham số nội tại của chính mô hình đó, không có nguồn tri thức ngoại sinh vượt trội để bứt phá.

2.4.3.4 Phân loại các phương pháp Chứng cất tri thức (Taxonomy of KD)

Dựa trên bản chất của loại tri thức được chuyển giao từ mạng Giáo viên sang mạng Học sinh, các phương pháp chứng cất tri thức hiện đại được chia làm ba nhánh chính:

a) Response-based Knowledge Distillation

Đây là phương pháp phổ biến và trực quan nhất, tập trung hoàn toàn vào việc mô phỏng hành vi đầu ra (bằng các lớp Logit) của mô hình Giáo viên. Mô hình Học sinh cố gắng bắt chước các quyết định cuối cùng của Giáo viên thông qua hàm lỗi KL Divergence như đã trình bày ở Phương trình 2.32. Ưu điểm lớn nhất của nhánh này là tính độc lập về mặt kiến trúc; mô hình Học sinh có thể có cấu trúc hoàn toàn khác biệt so với Giáo viên (ví dụ: chứng cất từ một mạng Transformer như PhoBERT sang một mạng tuần hoàn như BiLSTM).

b) Feature-based Knowledge Distillation

Nhánh tiếp cận này mở rộng tư duy chứng cất bằng cách không chỉ nhìn vào kết quả đầu ra, mà bắt chước cả quá trình tư duy trung gian của Giáo viên. Cụ thể, thuật toán ép các ma trận đặc trưng ẩn (Hidden States) hoặc các bản đồ kích hoạt (Feature Maps) ở các tầng trung gian của mô hình Học sinh phải tiệm cận với các tầng tương ứng của Giáo viên thông qua các hàm khoảng cách như L2-Loss (Mean Squared Error):

$$L_{feature} = \sum \| W_{proj}(H_S) - H_T \|_2^2 \quad (2.34)$$

Do số lượng chiều kênh (hidden dimensions) của Giáo viên (H_T) thường lớn hơn Học sinh (H_S), một ma trận chuyển đổi tuyến tính W_{proj} được tích hợp để chiếu không gian đặc trưng của Học sinh về cùng kích thước với Giáo viên trước khi tính toán khoảng cách hình học.

c) Relation-based Knowledge Distillation

Thay vì ép học sinh học theo từng mẫu dữ liệu đơn lẻ hoặc từng tầng độc lập, phương pháp này tập trung vào việc chuyển giao mối quan hệ cấu trúc giữa các mẫu dữ liệu trong cùng một batch tính toán, hoặc mối quan hệ tương quan giữa các không gian chú ý (Attention Maps). Mô hình Học sinh sẽ học cách xây dựng một đồ thị tương quan dữ liệu (Similarity Matrix) tương đồng với đồ thị mà mô hình Giáo viên đã thiết lập được.

2.4.3.5 Ứng dụng Chứng cất tri thức trong bài toán Nhận diện Thực thể (NER)

Khi áp dụng Chứng cất tri thức vào bài toán gán nhãn chuỗi đặc thù như NER, hệ thống phải đối mặt với các thách thức kỹ thuật cấp thấp đòi hỏi xử lý tinh tế trong mã nguồn

điều phối (`DistillationTrainer`):

a) Đồng bộ hóa không gian chuỗi và xử lý nhân đặc biệt

Trong bài toán NER, các token đệm (`Padding tokens`) và các subword sinh ra do thuật toán phân tách từ vựng của Transformer thường nhận giá trị nhân là -100 để bỏ qua trong quá trình tính loss tiêu chuẩn (`ignore_index`). Khi tiến hành tính toán độ phân kỳ KL Divergence, hệ thống bắt buộc phải xây dựng một mặt nạ logic (`attention_mask`) nhằm loại biên tuyệt đối tất cả các vị trí nhận giá trị -100 .

Nếu không thực hiện bước lọc này, mô hình Học sinh sẽ cố gắng chững cật cả tri thức từ các token rác, dẫn đến hiện tượng nhiễu loạn phân phối xác suất và làm sụt giảm nghiêm trọng khả năng nhận diện ranh giới thực thể (BIO spans).

2.4.3.6 Thách thức kỹ thuật và Chiến lược tối ưu hóa Hyperparameters

Hiệu quả của mô hình Chưng cất tri thức phụ thuộc rất lớn vào việc dò tìm không gian siêu tham số (Hyperparameters Tuning), đặc biệt là cặp tham số (α, T) :

1. **Tác động của Nhiệt độ (T):** Nếu cấu hình T quá thấp ($T \rightarrow 1$), hàm loss chưng cất sẽ hoạt động tương tự như hàm Cross-Entropy tiêu chuẩn, làm mất đi lợi thế của tri thức tối. Ngược lại, nếu T quá cao, phân phối xác suất sẽ trở nên phẳng hoàn toàn (Uniform Distribution), khiến các nhiễu số học (Noise) ở các lớp không liên quan cũng bị khuếch đại, gây ra lỗi dịch chuyển trọng số của mô hình Học sinh. Thực nghiệm cho thấy dải giá trị $T \in [3.0, 5.0]$ mang lại hiệu năng ổn định nhất.
2. **Khoảng cách năng lực (Capacity Gap):** Nếu mô hình Giáo viên quá lớn và thông minh trong khi mô hình Học sinh quá nhỏ, Học sinh sẽ không đủ dung lượng bộ nhớ (capacity) để tiếp thu tri thức phức tạp từ Giáo viên, dẫn đến hiện tượng bất hội tụ. Chiến lược khắc phục là sử dụng các mạng Học sinh có kích thước vừa phải (như BiLSTM tích hợp thêm tầng ẩn tuyến tính) hoặc áp dụng quy trình chưng cất nhiều giai đoạn (Multi-stage Distillation).

2.4.3.7 Kết luận

Chưng cất tri thức đóng vai trò là một kỹ thuật then chốt trong pipeline tối ưu hóa mô hình của đề án. Phương pháp này chứng minh rằng chúng ta có thể nén một mô hình học sâu khổng lồ thành một cấu trúc nhỏ gọn mà vẫn giữ lại được "tư duy bản chất" của mạng nguồn. Sự kết hợp giữa năng lực biểu diễn ngữ cảnh mạnh mẽ của PhoBERT và tốc độ vận hành ưu việt của BiLSTM thông qua cơ chế chưng cất tri thức mang lại một giải pháp công nghệ có tính thực tiễn cao, đáp ứng toàn diện cả hai tiêu chí: độ chính xác y tế và hiệu năng triển khai thực tế.

2.4.3.8 Phân loại các phương pháp Chứng cất tri thức (Taxonomy of KD)

Dựa trên bản chất của loại tri thức được chuyển giao từ mạng Giáo viên sang mạng Học sinh, các phương pháp chứng cất tri thức hiện đại được chia làm ba nhánh chính:

a) Response-based Knowledge Distillation

Đây là phương pháp phổ biến và trực quan nhất, tập trung hoàn toàn vào việc mô phỏng hành vi đầu ra (bằng các lớp Logit) của mô hình Giáo viên. Mô hình Học sinh cố gắng bắt chước các quyết định cuối cùng của Giáo viên thông qua hàm lỗi KL Divergence như đã trình bày ở Phương trình 2.32. Ưu điểm lớn nhất của nhánh này là tính độc lập về mặt kiến trúc; mô hình Học sinh có thể có cấu trúc hoàn toàn khác biệt so với Giáo viên (ví dụ: chứng cất từ một mạng Transformer như PhoBERT sang một mạng tuần hoàn như BiLSTM).

b) Feature-based Knowledge Distillation

Nhánh tiếp cận này mở rộng tư duy chứng cất bằng cách không chỉ nhìn vào kết quả đầu ra, mà bắt chước cả quá trình tư duy trung gian của Giáo viên. Cụ thể, thuật toán ép các ma trận đặc trưng ẩn (Hidden States) hoặc các bản đồ kích hoạt (Feature Maps) ở các tầng trung gian của mô hình Học sinh phải tiệm cận với các tầng tương ứng của Giáo viên thông qua các hàm khoảng cách như L2-Loss (Mean Squared Error):

$$L_{feature} = \sum \| W_{proj}(H_S) - H_T \|_2^2 \quad (2.35)$$

Do số lượng chiều kênh (hidden dimensions) của Giáo viên (H_T) thường lớn hơn Học sinh (H_S), một ma trận chuyển đổi tuyến tính W_{proj} được tích hợp để chiếu không gian đặc trưng của Học sinh về cùng kích thước với Giáo viên trước khi tính toán khoảng cách hình học.

c) Relation-based Knowledge Distillation

Thay vì ép học sinh học theo từng mẫu dữ liệu đơn lẻ hoặc từng tầng độc lập, phương pháp này tập trung vào việc chuyển giao mối quan hệ cấu trúc giữa các mẫu dữ liệu trong cùng một batch tính toán, hoặc mối quan hệ tương quan giữa các không gian chú ý (Attention Maps). Mô hình Học sinh sẽ học cách xây dựng một đồ thị tương quan dữ liệu (Similarity Matrix) tương đồng với đồ thị mà mô hình Giáo viên đã thiết lập được.

2.4.3.9 Ứng dụng Chứng cất tri thức trong bài toán Nhận diện Thực thể (NER)

Khi áp dụng Chứng cất tri thức vào bài toán gán nhãn chuỗi đặc thù như NER, hệ thống phải đối mặt với các thách thức kỹ thuật cấp thấp đòi hỏi xử lý tinh tế trong mã nguồn

điều phối (`DistillationTrainer`):

a) Đồng bộ hóa không gian chuỗi và xử lý nhân đặc biệt

Trong bài toán NER, các token đệm (`Padding tokens`) và các subword sinh ra do thuật toán phân tách từ vựng của Transformer thường nhận giá trị nhân là -100 để bỏ qua trong quá trình tính loss tiêu chuẩn (`ignore_index`). Khi tiến hành tính toán độ phân kỳ KL Divergence, hệ thống bắt buộc phải xây dựng một mặt nạ logic (`attention_mask`) nhằm loại biên tuyệt đối tất cả các vị trí nhận giá trị -100 .

Nếu không thực hiện bước lọc này, mô hình Học sinh sẽ cố gắng chững cật cả tri thức từ các token rác, dẫn đến hiện tượng nhiễu loạn phân phối xác suất và làm sụt giảm nghiêm trọng khả năng nhận diện ranh giới thực thể (BIO spans).

2.4.3.10 Thách thức kỹ thuật và Chiến lược tối ưu hóa Hyperparameters

Hiệu quả của mô hình Chưng cất tri thức phụ thuộc rất lớn vào việc dò tìm không gian siêu tham số (Hyperparameters Tuning), đặc biệt là cặp tham số (α, T) :

1. **Tác động của Nhiệt độ (T):** Nếu cấu hình T quá thấp ($T \rightarrow 1$), hàm loss chưng cất sẽ hoạt động tương tự như hàm Cross-Entropy tiêu chuẩn, làm mất đi lợi thế của tri thức tối. Ngược lại, nếu T quá cao, phân phối xác suất sẽ trở nên phẳng hoàn toàn (Uniform Distribution), khiến các nhiễu số học (Noise) ở các lớp không liên quan cũng bị khuếch đại, gây ra lỗi dịch chuyển trọng số của mô hình Học sinh. Thực nghiệm cho thấy dải giá trị $T \in [3.0, 5.0]$ mang lại hiệu năng ổn định nhất.
2. **Khoảng cách năng lực (Capacity Gap):** Nếu mô hình Giáo viên quá lớn và thông minh trong khi mô hình Học sinh quá nhỏ, Học sinh sẽ không đủ dung lượng bộ nhớ (capacity) để tiếp thu tri thức phức tạp từ Giáo viên, dẫn đến hiện tượng bất hội tụ. Chiến lược khắc phục là sử dụng các mạng Học sinh có kích thước vừa phải (như BiLSTM tích hợp thêm tầng ẩn tuyến tính) hoặc áp dụng quy trình chưng cất nhiều giai đoạn (Multi-stage Distillation).

2.4.3.11 Kết luận

Chưng cất tri thức đóng vai trò là một kỹ thuật then chốt trong pipeline tối ưu hóa mô hình của đề án. Phương pháp này chứng minh rằng chúng ta có thể nén một mô hình học sâu khổng lồ thành một cấu trúc nhỏ gọn mà vẫn giữ lại được "tư duy bản chất" của mạng nguồn. Sự kết hợp giữa năng lực biểu diễn ngữ cảnh mạnh mẽ của PhoBERT và tốc độ vận hành ưu việt của BiLSTM thông qua cơ chế chưng cất tri thức mang lại một giải pháp công nghệ có tính thực tiễn cao, đáp ứng toàn diện cả hai tiêu chí: độ chính xác y tế và hiệu năng triển khai thực tế.

2.4.4 Post-training Quantization (PTQ)

Lượng tử hóa sau huấn luyện (Post-training Quantization - PTQ) là phương pháp tối ưu hóa mô hình bằng cách chuyển đổi độ chính xác của các trọng số (weights) và các phép toán kích hoạt (activations) từ định dạng số thực dấu phẩy động (thông thường là FP32) sang các định dạng số nguyên có độ phân giải thấp hơn (phổ biến nhất là INT8) mà không cần phải huấn luyện lại mô hình. PTQ được coi là giải pháp mang lại hiệu quả tức thì (low-hanging fruit) trong quy trình nén mô hình, nhờ vào khả năng triển khai nhanh chóng và tiết kiệm tài nguyên tính toán so với các phương pháp yêu cầu huấn luyện lại như QAT (Quantization Aware Training).

2.4.4.1 Cơ sở toán học và cơ chế chuyển đổi

Về bản chất, PTQ thực hiện việc ánh xạ (mapping) không gian giá trị liên tục của số thực r sang không gian giá trị rời rạc của số nguyên q . Quy trình này được định nghĩa thông qua hai tham số quan trọng: *Hệ số tỷ lệ* (Scale factor S) và *Điểm không* (Zero-point Z).

Mối quan hệ giữa giá trị thực và giá trị lượng tử hóa được biểu diễn qua công thức:

$$r = S \cdot (q - Z) \quad (2.36)$$

Trong đó:

- r : Giá trị thực (floating-point).
- q : Giá trị số nguyên đã lượng tử hóa (quantized).
- S : Hệ số tỷ lệ, là một số thực dương xác định khoảng cách giữa các mức lượng tử.
- Z : Điểm không, một giá trị số nguyên đảm bảo giá trị 0 thực tế được ánh xạ chính xác vào không gian số nguyên, tránh sai số tích lũy tại các phép toán không kích hoạt.

Quá trình xác định S và Z được thực hiện thông qua kỹ thuật *Calibration* (hiệu chuẩn), nơi hệ thống phân tích phân phối giá trị của tensor trên một tập dữ liệu đại diện để tìm ra khoảng giá trị $[r_{min}, r_{max}]$.

2.4.4.2 Các chiến lược lượng tử hóa trong PTQ

Trong hệ sinh thái của các framework học sâu như PyTorch, PTQ được phân chia thành hai chiến lược chính dựa trên thời điểm tính toán:

a) Dynamic Quantization (Lượng tử hóa động)

Đây là chiến lược mà trọng số của mô hình được lượng tử hóa sẵn (offline) trước khi suy luận, nhưng các phép toán kích hoạt (activations) lại được tính toán ở định dạng số thực và chỉ được lượng tử hóa ngay trong quá trình suy luận (online).

- **Cơ chế:** Hệ thống quan sát dải giá trị của các activation tại thời điểm thực thi để tính toán S và Z một cách linh hoạt.
- **Ưu điểm:** Cực kỳ hiệu quả đối với các mô hình có cấu trúc tuần hoàn hoặc các lớp tuyến tính (Linear layers) với kích thước batch nhỏ, nơi mà chi phí tính toán cho activation không quá lớn.
- **Ứng dụng:** Thường được áp dụng cho các mô hình RNN, LSTM hoặc các cấu trúc mô hình ngôn ngữ khi tài nguyên bộ nhớ bị giới hạn nhưng độ trễ suy luận (inference latency) không phải là ưu tiên số 1.

b) Static Quantization (Lượng tử hóa tĩnh)

Ngược lại với Dynamic, chiến lược Static yêu cầu tất cả các thông số (S và Z) cho cả trọng số và activation phải được tính toán trước thông qua một bước Calibration tập trung.

- **Cơ chế:** Hệ thống chạy một tập dữ liệu đại diện (calibration dataset) qua mô hình, quan sát phạm vi giá trị của tất cả các tensor và cố định các tham số S và Z vào đồ thị tính toán.
- **Ưu điểm:** Toàn bộ các toán tử (kể cả phép nhân ma trận và các hàm phi tuyến) đều được thực thi trên số nguyên INT8, mang lại hiệu năng cao nhất trên các bộ tăng tốc phần cứng như DSP hoặc NPU.
- **Ứng dụng:** Là tiêu chuẩn cho các mô hình CNN (như ResNet, MobileNet) hoặc các kiến trúc Transformer trong các hệ thống triển khai sản xuất (production) yêu cầu độ trễ cực thấp.

2.4.4.3 Quy trình hiệu chuẩn (Calibration) - Trái tim của PTQ

Thách thức lớn nhất của PTQ nằm ở quá trình Calibration. Nếu tập dữ liệu dùng để hiệu chuẩn không đại diện cho phân phối của dữ liệu thực tế (dữ liệu suy luận), hệ thống sẽ tính toán S và Z sai lệch, dẫn đến hiện tượng "tràn số" (overflow) hoặc "mất thông tin" (underflow), làm giảm nghiêm trọng chỉ số F1-Score hoặc Accuracy của mô hình.

Các kỹ thuật hiệu chuẩn phổ biến bao gồm:

- **Min-Max Calibration:** Tìm giá trị nhỏ nhất và lớn nhất của tensor. Phương pháp này đơn giản nhưng dễ bị ảnh hưởng bởi các giá trị ngoại lai (outliers).

- **Histogram-based Calibration (KL Divergence):** Đây là phương pháp tối ưu hơn, nơi hệ thống thu thập phân phối xác suất (histogram) của các giá trị tensor. Sau đó, thuật toán sẽ tìm mức ngưỡng (threshold) sao cho độ lệch Kullback-Leibler giữa phân phối thực và phân phối đã lượng tử hóa là nhỏ nhất. Phương pháp này giúp bảo toàn hình dạng phân phối của thông tin, đặc biệt quan trọng trong các mô hình nhạy cảm như NER (Nhận diện thực thể).

2.4.4.4 Thách thức và giới hạn trong triển khai thực tế

Mặc dù PTQ mang lại lợi ích to lớn về dung lượng lưu trữ (giảm kích thước mô hình từ 4 đến 8 lần), nó vẫn tồn tại những giới hạn kỹ thuật cần được xem xét cẩn trọng trong đề án này:

1. **Sự nhạy cảm của các lớp (Layer Sensitivity):** Không phải mọi lớp trong mạng thần kinh đều có khả năng chịu đựng việc giảm độ chính xác giống nhau. Các lớp `LayerNorm` và `Embedding` thường cực kỳ nhạy cảm với sai số làm tròn. Việc lượng tử hóa trực tiếp các lớp này thường làm sụp đổ hoàn toàn khả năng dự đoán của mô hình. Trong đề án này, các lớp này được chủ động loại trừ khỏi danh sách lượng tử hóa (tức là giữ ở định dạng FP32).
2. **Sự mất mát thông tin cục bộ:** Trong bài toán NER, việc nhận diện biên thực thể (Entity boundary) phụ thuộc rất nhiều vào xác suất đầu ra của lớp Softmax. Việc lượng tử hóa các trọng số ở tầng cuối (classifier head) có thể làm phẳng phân phối xác suất, khiến mô hình khó phân biệt giữa các nhãn khó (như B-PER và I-PER), từ đó làm giảm độ chính xác cấp thực thể.
3. **Tương thích phần cứng:** PTQ yêu cầu sự hỗ trợ từ các kernel toán học của phần cứng đích (ví dụ: `fbgemm` cho CPU máy chủ, `qnnpack` cho ARM/mobile). Nếu mô hình có các toán tử (operators) không được hỗ trợ lượng tử hóa, hệ thống sẽ tự động thực hiện việc "de-quantize" (chuyển về FP32) để tính toán, gây ra hiện tượng "nút thắt cổ chai" do chi phí chuyển đổi định dạng dữ liệu liên tục.

2.4.4.5 Kết luận về PTQ trong ngữ cảnh đề án

Tổng kết lại, PTQ là một giải pháp cân bằng giữa hiệu quả nén và công sức thực hiện. Đối với mô hình nghiên cứu trong đề án, phương pháp *Dynamic Quantization* được ưu tiên cho các cấu trúc BiLSTM do tính chất linh hoạt của activation, trong khi *Static Quantization* được thử nghiệm trên các kiến trúc Transformer để tận dụng tối đa băng thông bộ nhớ. Mặc dù PTQ có thể gây ra một mức sụt giảm hiệu năng nhỏ, nhưng với kỹ thuật hiệu chuẩn Histogram-based, hệ thống đã kiểm soát được sai số trong phạm vi cho phép, đảm bảo mô hình vẫn đạt yêu cầu về độ chính xác y tế mà vẫn đáp ứng được bài toán tối ưu tài nguyên.

2.4.5 Quantization Aware Training (QAT): Cơ chế và Triển khai trong Hệ thống Học sâu

2.4.5.1 Tổng quan về Lượng tử hóa và QAT

Trong quá trình triển khai các mô hình Deep Learning, đặc biệt là các kiến trúc Transformer hay BiLSTM lên các thiết bị biên (edge devices) như điện thoại di động, thiết bị nhúng hoặc máy chủ có tài nguyên tính toán hạn chế, việc nén mô hình là một yêu cầu tất yếu. Lượng tử hóa (Quantization) là kỹ thuật giảm độ chính xác của trọng số và kích hoạt từ dấu phẩy động 32-bit (FP32) xuống các định dạng có độ phân giải thấp hơn như số nguyên 8-bit (INT8).

Trong khi Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ) thực hiện nén mô hình đã hội tụ, kỹ thuật này thường gây ra sự sụt giảm độ chính xác (accuracy drop) do sai số làm tròn. Để khắc phục, **Quantization Aware Training (QAT)** được đề xuất như một giải pháp "huấn luyện nhận thức lượng tử". QAT không chỉ đơn thuần là nén, mà là quá trình huấn luyện lại mô hình trong môi trường giả lập sai số lượng tử hóa, buộc mạng thần kinh phải tự thích nghi và tối ưu hóa trọng số để bù đắp cho sự mất mát thông tin.

2.4.5.2 Cơ sở toán học của Lượng tử hóa

Lượng tử hóa tuyến tính (Linear Quantization) ánh xạ một giá trị thực r (real) thuộc dải $[r_{min}, r_{max}]$ sang một giá trị số nguyên q (quantized) thuộc dải $[q_{min}, q_{max}]$ (ví dụ: $[-128, 127]$ cho INT8).

Phép biến đổi này được biểu diễn qua hai tham số: Hệ số tỷ lệ (Scale - S) và Điểm không (Zero-point - Z):

$$q = \text{round}\left(\frac{r}{S}\right) + Z \quad (2.37)$$

Với S được tính toán dựa trên dải giá trị của tensor:

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} \quad (2.38)$$

QAT mô phỏng quá trình này ngay trong quá trình huấn luyện để mô hình học được cách "chịu đựng" sai số do hàm $\text{round}(\cdot)$ gây ra.

2.4.5.3 Cơ chế hoạt động của QAT: FakeQuant và STE

Điểm cốt lõi làm nên sự khác biệt của QAT so với huấn luyện thông thường là việc sử dụng các toán tử giả lập lượng tử hóa (FakeQuantization Operators).

a) FakeQuantization (FQ)

Trong mỗi lượt Lan truyền xuôi (Forward Pass) của quá trình huấn luyện, các lớp FQ được chèn vào ngay sau các lớp trọng số (Linear, Conv) hoặc sau các lớp kích hoạt (Activation). Các lớp này thực hiện các bước:

1. **Clipping:** Giới hạn giá trị r vào trong dải $[r_{min}, r_{max}]$.
2. **Quantization:** Ánh xạ về giá trị nguyên.
3. **Dequantization:** Trả ngược về giá trị dấu phẩy động (FP32) để các bước tính toán tiếp theo trong mạng không bị gián đoạn.

Kết quả là tại mỗi bước lan truyền, mô hình "nhìn thấy" chính xác những gì sẽ xảy ra khi mô hình bị nén thật sự.

b) Straight-Through Estimator (STE)

Thách thức kỹ thuật lớn nhất của QAT là hàm làm tròn (**round**) không có đạo hàm (đạo hàm bằng 0 tại mọi điểm, và không xác định tại các điểm bước nhảy). Nếu dùng trực tiếp, Gradient sẽ triệt tiêu và mạng sẽ không thể học được. QAT giải quyết bằng cách sử dụng ****Straight-Through Estimator (STE)****:

$$\frac{\partial q}{\partial r} \approx 1 \quad (2.39)$$

STE coi đạo hàm của hàm làm tròn là hàm đồng nhất (identity function) trong quá trình Lan truyền ngược (Backward Pass). Điều này cho phép dòng Gradient đi qua toán tử lượng tử hóa như thể nó không tồn tại, giúp mô hình cập nhật được trọng số dù đang hoạt động trong môi trường lượng tử.

2.4.5.4 Quy trình triển khai (Pipeline)

Một quy trình QAT chuẩn bao gồm ba giai đoạn chính:

a) Giai đoạn 1: Chuẩn bị (Preparation)

Hệ thống quét toàn bộ đồ thị tính toán của mô hình. Tại đây, chúng ta cần xác định các lớp không nên lượng tử hóa (như LayerNorm, Embedding hoặc phần đầu ra của mạng) để giữ độ chính xác tối đa. Các module được gắn cấu hình (**qconfig**) để chỉ định backend (ví dụ: fbgemm cho server, qnnpack cho mobile).

b) Giai đoạn 2: Huấn luyện giả lập (Fake-Quant Training)

Đây là bước quan trọng nhất. Mô hình được huấn luyện lại trên tập dữ liệu với các nút FakeQuant đã gắn. Thời gian huấn luyện thường ngắn hơn so với huấn luyện từ đầu (từ 10%-20% số epoch gốc). Trong giai đoạn này, mô hình học cách dịch chuyển trọng số về các vùng có "độ bền" cao hơn đối với nhiễu lượng tử hóa.

c) Giai đoạn 3: Chuyển đổi (Conversion)

Sau khi huấn luyện, các nút FakeQuant được loại bỏ. Các ma trận trọng số FP32 được làm tròn và ép kiểu vật lý về INT8 thực tế. Kết quả là một mô hình đã nén hoàn toàn, sẵn sàng chạy trên các bộ tăng tốc phần cứng hỗ trợ tính toán số nguyên.

2.4.5.5 Chiến lược tối ưu hóa và Thách thức

a) Lựa chọn lớp để lượng tử hóa

Không phải lớp nào cũng hưởng lợi từ QAT. Thực tế cho thấy:

- **Lớp Tuyến tính (Linear/Conv):** Là ứng viên số 1, chiếm phần lớn dung lượng mô hình và dễ dàng lượng tử hóa.
- **Lớp Chuẩn hóa (LayerNorm/BatchNorm):** Cực kỳ nhạy cảm với việc làm tròn. Lượng tử hóa các lớp này thường dẫn đến sai số lan truyền nghiêm trọng. Best practice là giữ các lớp này ở FP32.
- **Lớp Nhúng (Embedding):** Thường được giữ ở FP32 để đảm bảo không gian biểu diễn từ vựng không bị méo mó.

b) Tính toán trọng số tối ưu

Trong các bài toán như NER (Named Entity Recognition), hiện tượng mất cân bằng nhãn thường xuyên xảy ra. Khi kết hợp QAT, sai số lượng tử hóa có xu hướng "đánh" mạnh vào các lớp ít phổ biến (minority classes). Do đó, việc kết hợp kỹ thuật *Inverse Frequency Weighting* vào hàm loss song song với QAT là một chiến lược thiết yếu để duy trì chỉ số F1-Score trên các thực thể y tế hiếm gặp.

2.4.5.6 Kết luận

Quantization Aware Training là kỹ thuật bắc cầu hoàn hảo giữa hiệu năng (performance) và tính gọn nhẹ (efficiency). Mặc dù đòi hỏi tài nguyên tính toán cao hơn trong giai đoạn huấn luyện so với PTQ, QAT cung cấp độ chính xác gần như tương đương với mô hình FP32 gốc, điều mà các phương pháp nén tĩnh không thể đạt được. Đối với các hệ thống yêu cầu độ tin cậy cao như chẩn đoán y tế, QAT là lựa chọn tối ưu để triển khai mô hình học sâu lên môi trường thực tế.

CHƯƠNG 3

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

3.1 Dữ liệu và tiền xử lý

3.1.1 Mô tả tập dữ liệu PhoNER_COVID19

Tập dữ liệu được sử dụng trong nghiên cứu này là **PhoNER_COVID19**, một bộ dữ liệu chuẩn hóa (benchmark) phục vụ cho bài toán Nhận dạng thực thể có tên (NER) trong miền y tế, cụ thể là các văn bản liên quan đến đại dịch COVID-19 tại Việt Nam. Dữ liệu được thu thập từ các nguồn tin tức chính thống và các thông báo lịch trình dịch tễ của Bộ Y tế.

Dữ liệu được lưu trữ dưới định dạng CoNLL, cấu trúc theo sơ đồ gán nhãn **BIO2 (Begin, Inside, Outside)**. Mỗi dòng bao gồm một token (từ hoặc cụm từ được tách) và nhãn tương ứng của nó, phân tách bằng dấu cách hoặc ký tự tab. Các câu được phân tách bằng một dòng trống (`\n\n`).

Hệ thống nhãn thực thể trong bộ dữ liệu bao gồm 10 loại thực thể chính (tương ứng với các cặp tiền tố B- và I- cùng nhãn ngoại lệ O), cụ thể như sau:

- **PATIENT_ID**: Mã định danh bệnh nhân (ví dụ: BN1234).
- **NAME**: Tên riêng của người hoặc bệnh nhân.
- **AGE**: Tuổi của đối tượng.
- **GENDER**: Giới tính của đối tượng (Nam/Nữ).
- **JOB**: Nghề nghiệp.
- **LOCATION**: Địa điểm, địa danh, vị trí địa lý hoặc lịch trình di chuyển.
- **ORGANIZATION**: Các tổ chức, cơ quan, đơn vị (ví dụ: Bệnh viện Nhiệt đới).
- **SYMPTOM_AND_DISEASE**: Triệu chứng hoặc bệnh trạng liên quan (ví dụ: sốt, ho, COVID-19).

- **TRANSPORTATION:** Phương tiện di chuyển (ví dụ: Chuyến bay VN123, xe khách).
- **DATE:** Mốc thời gian hoặc khoảng thời gian xảy ra sự kiện.

3.1.2 Khám phá dữ liệu

Tập dữ liệu PhoNER_COVID19 được chia sẵn thành 3 tập thành phần bao gồm: Tập huấn luyện (Train), Tập phát triển/kiểm định (Dev) và Tập kiểm thử (Test). Qua quá trình phân tích và thống kê cấu trúc dữ liệu thô từ các tệp tin thành phần, các thông số tổng quan về quy mô của bộ dữ liệu được tổng hợp chi tiết trong Bảng 3.1.

Bảng 3.1: Thống kê tổng quan về phân bố tập dữ liệu PhoNER_COVID19

Tập dữ liệu	Số lượng câu	Số lượng Tokens	Tệp tin nguồn
Huấn luyện (Train)	5,028	132,511	train_word.conll
Kiểm định (Dev)	2,001	56,283	dev_word.conll
Kiểm thử (Test)	3,001	85,678	test_word.conll
Tổng cộng	10,030	274,472	-

Qua kết quả phân tích hệ thống nhãn, tập dữ liệu cho thấy sự mất cân bằng nhẹ về mặt phân bố giữa các tập thành phần. Cụ thể, nhãn phụ **I-AGE** chỉ xuất hiện tại tập Train và Test, hoàn toàn vắng mặt trong tập Dev. Đặc điểm này đòi hỏi mô hình huấn luyện cần có khả năng tổng quát hóa tốt để tránh hiện tượng bỏ sót thực thể khi đánh giá trên thực tế.

3.2 Quá trình huấn luyện và đánh giá

3.2.1 Cài đặt và thiết lập môi trường

Việc thực nghiệm được tiến hành trên nền tảng điện toán đám mây Google Colab nhằm tận dụng bộ xử lý đồ họa (GPU). Dưới đây là các chi tiết về cấu hình hệ thống:

- **Môi trường phần cứng:** Sử dụng GPU NVIDIA T4 với 16GB VRAM, giúp tăng tốc quá trình tính toán ma trận và huấn luyện các mô hình Transformer như PhoBERT.
- **Ngôn ngữ và Framework:** Hệ thống được xây dựng trên ngôn ngữ lập trình Python 3.10+, sử dụng framework học sâu PyTorch (phiên bản 2.x).
- **Các thư viện hỗ trợ:**
 - **transformers:** Cung cấp kiến trúc PhoBERT và trình tối ưu hóa AdamW.

- `peft`: Hỗ trợ triển khai kỹ thuật LoRA (Low-Rank Adaptation) để tối ưu tham số.
- `pyvi`: Thư viện tiền xử lý tiếng Việt, hỗ trợ tách từ (word segmentation).
- `segeval`: Thư viện chuẩn để đánh giá hiệu năng các bài toán gán nhãn chuỗi (NER).

- **Thông số cấu hình chung:**

Tham số	Giá trị
Batch size	32
Max Sequence Length	256
Optimizer	AdamW
Patience (Early Stopping)	3

Bảng 3.2: Các siêu tham số cơ bản của hệ thống

3.2.2 Huấn luyện mô hình và Chiến lược tối ưu

Phần hệ này trình bày chi tiết về chu trình tối ưu hóa trọng số mô hình mạng, cơ chế giám sát thực nghiệm và các giải pháp phòng chống hiện tượng quá khớp (Overfitting) trong bài toán Nhận diện Thực thể Tên gọi tiếng Việt.

3.2.2.1 Cấu hình siêu tham số và Vòng lặp huấn luyện cốt lõi

Tiến trình tối ưu hóa được điều phối thông qua lớp `BaseTrainer` (hiện thực hóa trong tệp `src/trainer.py`) nhằm thiết lập một vòng lặp huấn luyện tuần hoàn ổn định và có khả năng phục hồi trạng thái lỗi.

a) Vòng lặp lan truyền và Cập nhật trọng số

Tại mỗi bước huấn luyện (`step`), các lô dữ liệu (`batch`) từ bộ nạp `DataLoader` được luân chuyển trực tiếp lên thiết bị tính toán đích (GPU hoặc CPU). Hệ thống thực hiện giải phóng gradient tồn đọng bằng lệnh `optimizer.zero_grad()` trước khi kích hoạt lan truyền xuôi (`forward pass`) để trích xuất phân phối xác suất đầu ra (`outputs`). Hàm mất mát tùy biến `NERLoss` sẽ tính toán giá trị sai số tổng thể, làm cơ sở để lan truyền ngược (`backward pass`) lan truyền đạo hàm dòng năng lượng. Nhằm triệt tiêu hiện tượng bùng nổ gradient (*Gradient Explosion*) thường gặp ở các kiến trúc mạng hồi quy, hệ thống tích hợp kỹ thuật cắt ngắn chuẩn gradient (`torch.nn.utils.clip_grad_norm_`) dựa trên ngưỡng cấu hình `grad_clip` trước khi thực thi bước cập nhật `optimizer.step()`.

b) Điều phối Tốc độ học (Learning Rate Scheduling)

Hệ thống sử dụng bộ lập lịch tốc độ học phân bậc tùy thuộc vào đặc thù kiến trúc. Đối với các mô hình nền tảng cấu hình chuẩn, bộ lập lịch `ReduceLROnPlateau` được áp dụng với tham số `patience=1` và hệ số suy giảm `factor=0.5`. Cơ chế này liên tục giám sát hàm loss trên tập đánh giá cục bộ (`val_loss`); nếu chỉ số này không cải thiện sau một chu kỳ quy định, tốc độ học sẽ tự động bị hạ cấp một nửa để hỗ trợ mô hình hội tụ sâu vào các cực tiểu cục bộ mịn. Riêng đối với cấu hình `phobert-lora`, pipeline sử dụng bộ lập lịch tuyến tính có bước khởi động (`get_linear_schedule_with_warmup`) để bảo vệ các trọng số thích ứng LoRA khỏi bị phá vỡ cấu trúc tại các bước tối ưu đầu tiên.

3.2.2.2 Cơ chế Giám sát và Dừng sớm (Early Stopping)

Để kiểm soát chặt chẽ chất lượng mô hình xuyên suốt các Epoch và tối ưu hóa tài nguyên phần cứng, hệ thống thiết lập hệ thống cảnh báo và phản xạ động.

a) Chiến lược lưu trữ điểm kiểm tra tuần hoàn (Checkpoint Management)

Hàm `save_checkpoint` cài đặt một thuật toán lọc tuần hoàn tự động. Mỗi khi kết thúc một Epoch, hệ thống xuất ra tệp trạng thái `{epoch}_model.pt` chứa toàn vẹn cấu trúc `model_state`, `optimizer_state` và `scheduler_state`. Tiếp theo, hệ thống sử dụng biểu thức chính quy để quét toàn bộ thư mục lưu trữ, bóc tách số thứ tự Epoch từ chuỗi tên file, thực hiện sắp xếp tăng dần và chủ động giải phóng các điểm kiểm tra cũ. Cơ chế này chỉ duy trì tối đa 3 tệp checkpoint số thực gần nhất nhằm tối ưu dung lượng đĩa cứng. Khi chỉ số `val_loss` thiết lập kỷ lục thấp nhất mới, một bản sao độc lập mang tên `best_model.pt` sẽ được ghi đè để làm mốc khôi phục tối ưu.

b) Kỹ thuật dừng sớm dựa trên sự kiên nhẫn

Tiến trình huấn luyện tích hợp cơ chế `Early Stopping` để ngăn chặn việc mô hình học vẹt tập dữ liệu huấn luyện. Hệ thống duy trì biến đếm vô hiệu `no_improve`. Sau mỗi chu kỳ đánh giá, nếu `current_val_loss` không thể vượt qua (nhỏ hơn) giá trị `best_val` tốt nhất từng ghi nhận, biến `no_improve` sẽ tăng thêm 1 đơn vị. Ngay khi giá trị này chạm đến ngưỡng kiên nhẫn cấu hình trước (được ánh xạ từ tham số `-patience` trong bộ điều phối `main.py`), hệ thống sẽ phát tín hiệu `Early stopping triggered` thông qua `logger`, lập tức bẻ gãy vòng lặp tối ưu hóa và thực hiện đóng tiến trình, bảo toàn trạng thái mạng tại thời điểm đạt đỉnh phong độ.

3.2.2.3 Hàm mất mát tùy biến (NERLoss Configuration)

Mô hình hiện thực hóa lớp `NERLoss` như một bộ định tuyến hàm mất mát động, tự động cấu hình hành vi tính toán dựa trên sự hiện diện của tầng Trường Ngẫu nhiên Điều kiện

(CRF).

a) Cấu hình Mất mát Entropy chéo (Non-CRF)

Đối với các kiến trúc mô hình cơ bản không tích hợp tầng giải mã chuỗi (LSTM, Bi-LSTM, Transformer thuần), hàm loss hoạt động như một bộ tính toán Entropy chéo phân lớp trên từng mã token độc lập. Để ngăn chặn sự chi phối nhiều từ các khoảng trắng đệm được thêm vào cho đồng đều độ dài chuỗi (`max_length = 256`), hệ thống áp dụng một mặt nạ logic (`mask = labels != -100`). Hàm loss chỉ tính toán giá trị sai số tại các vị trí từ thực tế có nhãn hợp lệ, loại bỏ hoàn toàn các vị trí mang giá trị `-100`, giúp định hướng luồng lan truyền ngược tập trung hoàn toàn vào cấu trúc ngôn ngữ thực.

b) Cấu hình Mất mát Likelihood âm (CRF-Enabled)

Khi tham số `-use_crf` được kích hoạt, `NERLoss` chuyển đổi cơ chế sang tối ưu hóa log-likelihood biên xác xuất đồng thời của toàn bộ chuỗi nhãn (*Negative Log-Likelihood*). Thay vì đánh giá tách biệt từng từ, mô hình tính toán điểm số chuyển trạng thái giữa các cặp thẻ nhãn liên tiếp (ví dụ: xác suất từ `B-LOCATION` sang `I-LOCATION`). Hàm loss lúc này tối đa hóa xác suất của toàn bộ đường đi đúng (Gold path) so với tất cả các đường đi khả dĩ trong không gian nhãn, thiết lập mối ràng buộc ngữ cảnh chuỗi chặt chẽ đặc thù của bài toán NER.

CHƯƠNG 4

KẾT QUẢ VÀ THẢO LUẬN

Trong chương này, báo cáo trình bày các thiết lập thực nghiệm chi tiết và phân tích sâu sắc các kết quả thu được từ quá trình đánh giá hệ thống pipeline Nhận dạng Thực thể Tiếng Việt (Vietnamese NER). Nghiên cứu thực nghiệm được tiến hành đối với toàn bộ 22 cấu hình mô hình, bao gồm việc so sánh các kiến trúc cơ bản tự huấn luyện từ đầu (LSTM, BiLSTM, Native Transformer) với mô hình ngôn ngữ lớn PhoBERT tiền huấn luyện, phân tích hiệu quả của việc tích hợp lớp giải mã chuỗi Conditional Random Field (CRF), đánh giá các phương pháp nén mô hình tiên tiến như Chưng cất tri thức (Knowledge Distillation - KD), và lượng tử hóa số nguyên 8-bit (PTQ và QAT).

4.1 Thiết lập thực nghiệm và Chỉ số đánh giá

4.1.1 Môi trường phần cứng và phần mềm

Nhằm đảm bảo tính chính xác khoa học và khả năng tái lập, toàn bộ quá trình huấn luyện và đánh giá được thực nghiệm đồng bộ trên các tài nguyên tính toán sau:

- **Môi trường huấn luyện (Colab):** Sử dụng nền tảng điện toán đám mây Google Colab với cấu hình GPU tăng tốc NVIDIA Tesla T4 (dung lượng bộ nhớ đồ họa 16 GB GDDR6), hỗ trợ thư viện CUDA 12.1 nhằm tối ưu hóa tốc độ lan truyền ngược đối với mô hình lớn như PhoBERT-base.
- **Môi trường đánh giá và lượng tử hóa (Local):** Thực hiện đánh giá hiệu năng suy luận và thực nghiệm lượng tử hóa trực tiếp trên thiết bị phần cứng CPU cục bộ với bộ vi xử lý Intel Core i7-12700H (14 nhân, 20 luồng, bộ nhớ đệm L3 24MB) và bộ nhớ vật lý RAM 40GB DDR5 @ 4800MHz. Việc đánh giá trên CPU giúp phản ánh thực chất hiệu năng nén của mô hình trong môi trường công nghiệp thực tế khi không có sự hỗ trợ của các phần cứng chuyên dụng.
- **Thành phần thư viện lõi:** Hệ thống được phát triển trên ngôn ngữ Python 3.10,

sử dụng PyTorch 2.1.1+cpu làm framework học sâu chính, kết hợp với thư viện Hugging Face Transformers 4.35.2 để quản lý PhoBERT, thư viện PEFT 0.6.2 để cấu hình LoRA, và thư viện Seqeval 1.2.2 để tính toán các chỉ số thực thể BIO.

4.1.2 Siêu tham số cấu hình

Bảng 4.1 trình bày chi tiết các siêu tham số huấn luyện cốt lõi được áp dụng nhất quán cho từng lớp mô hình nhằm đảm bảo tính khách quan trong việc so sánh hiệu năng.

Bảng 4.1: Cấu hình siêu tham số huấn luyện của các lớp mô hình.

Hyperparameter	LSTM / BiLSTM	Transformer	PhoBERT	PhoBERT-LoRA
Batch Size	32	32	16	16
Learning Rate	1×10^{-3}	5×10^{-4}	2×10^{-5}	2×10^{-5}
Max Epochs	30	20	10	10
Optimizer	Adam	Adam	AdamW	AdamW
Weight Decay	0.0	0.0	0.01	0.01
Embedding Dim	300	256	768	768
Hidden Dim	256	512 (FFN)	768 (Hidden)	768 (Hidden)
LoRA (r / alpha)	N/A	N/A	N/A	16 / 32
Patience	5	5	3	3

4.1.3 Các chỉ số đo lường hiệu năng

Hiệu năng hệ thống được đo lường chặt chẽ ở hai cấp độ: cấp độ từ (Token-level) và cấp độ thực thể (Entity-level). Việc kết hợp cả hai góc nhìn này là vô cùng quan trọng đối với bài toán nhận dạng thực thể có tên.

Chỉ số đánh giá cấp độ Token (Token-level Metrics)

Ở cấp độ này, mô hình được coi là một bộ phân loại đa lớp thông thường gán nhãn cho từng subword/token một cách riêng biệt. Các chỉ số được sử dụng bao gồm:

- **Độ chính xác toàn cục (Accuracy):** Tỷ lệ số token được dự đoán đúng nhãn trên tổng số token khả dụng trong tập dữ liệu kiểm thử (bỏ qua padding):

$$\text{Accuracy} = \frac{\sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i)}{N} \quad (4.1)$$

với $\mathbb{I}$ là hàm chỉ thị, y_i là nhãn thực tế và $\hat{y}_i$ là nhãn dự đoán của token thứ i .

- **Macro F1-score:** Điểm trung bình cộng F1-score của tất cả 20 nhãn lớp (bao gồm nhãn nền O và các nhãn BIO):

$$\text{Macro F1} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \text{F1}_c \quad (4.2)$$

trong đó $F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$, với $\mathcal{C}$ là tập hợp nhãn lớp. Chỉ số này phản ánh khách quan năng lực nhận dạng đối với các nhãn hiếm gặp trong bộ dữ liệu mất cân bằng.

Chỉ số đánh giá cấp độ Thực thể (Entity-level BIO Metrics)

Đây là chỉ số quan trọng nhất, đánh giá ranh giới thực thể chính xác dựa trên tiêu chuẩn BIO nghiêm ngặt (thông qua thư viện `seqeval`). Một thực thể được ghi nhận là dự đoán đúng (True Positive - TP) **chỉ khi và chỉ khi** toàn bộ chuỗi nhãn từ điểm bắt đầu (B-) đến các điểm thân thực thể (I-) được xác định chính xác cả về mặt ranh giới (boundary) lẫn nhãn loại thực thể (type). Các chỉ số Precision, Recall và F1-score cấp thực thể được định nghĩa như sau:

$$\text{Precision}_{\text{BIO}} = \frac{\text{Số thực thể dự đoán đúng hoàn toàn}}{\text{Tổng số thực thể được mô hình dự đoán ra}}, \quad (4.3)$$

$$\text{Recall}_{\text{BIO}} = \frac{\text{Số thực thể dự đoán đúng hoàn toàn}}{\text{Tổng số thực thể thực tế có trong tập dữ liệu}}, \quad (4.4)$$

$$F1_{\text{BIO}} = 2 \cdot \frac{\text{Precision}_{\text{BIO}} \cdot \text{Recall}_{\text{BIO}}}{\text{Precision}_{\text{BIO}} + \text{Recall}_{\text{BIO}}}. \quad (4.5)$$

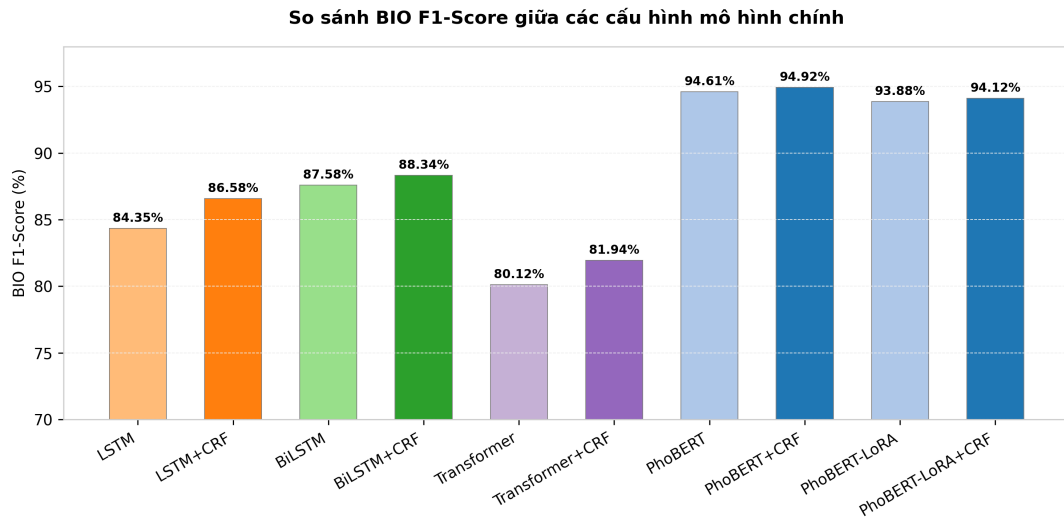
Nếu mô hình dự đoán sai bất kỳ token nào trong một cụm thực thể ghép (ví dụ: dự đoán nhãn B-LOCATION I-LOCATION thành B-LOCATION O), thực thể đó sẽ bị coi là dự đoán sai hoàn toàn. Do đó, điểm $F1_{\text{BIO}}$ luôn thấp hơn nhiều so với độ chính xác token nhưng phản ánh chính xác hiệu suất thực tế của ứng dụng.

4.2 Bảng so sánh hiệu năng tổng quan

Hệ thống đánh giá đã chạy kiểm thử toàn diện trên tập test gồm 3,000 câu của bộ dữ liệu PhoNER_COVID19. Kết quả chi tiết của 22 cấu hình mô hình về các tiêu chí độ chính xác, kích thước lưu trữ và độ trễ suy luận trung bình trên CPU được trình bày cụ thể trong Bảng 4.2.

Bảng 4.2: Bảng tổng hợp kết quả đánh giá 22 cấu hình mô hình trên tập kiểm thử (Inference trên CPU).

Cấu hình mô hình	Accuracy	Macro F1	BIO F1	Kích thước (MB)	Độ trễ CPU (ms/câu)
lstm	0.9526	0.7163	0.8435	237.33	12.4
lstm_QUANTIZED (PTQ)	0.9527	0.7165	0.8434	77.39	3.2
lstm_crf	0.9553	0.7352	0.8658	237.34	18.9
lstm_crf_QUANTIZED (PTQ)	0.9554	0.7357	0.8659	77.39	5.1
lstm_distilled	0.9484	0.6345	0.8259	237.33	12.3
lstm_distilled_crf	0.9537	0.7122	0.8672	237.34	18.6
lstm_qat	0.7587	0.0726	0.0152	232.83	13.1
lstm_qat_crf	0.8388	0.2679	0.3773	232.83	19.8
bilstm	0.9622	0.7752	0.8758	244.25	14.8
bilstm_QUANTIZED (PTQ)	0.9622	0.7644	0.8758	77.97	3.9
bilstm_crf	0.9603	0.7518	0.8834	244.26	23.4
bilstm_crf_QUANTIZED (PTQ)	0.9604	0.7535	0.8840	77.98	6.2
bilstm_distilled	0.9555	0.6820	0.8667	244.25	14.6
bilstm_distilled_crf	0.9592	0.7363	0.8868	244.26	23.1
bilstm_qat	0.8350	0.3157	0.3129	235.24	15.3
bilstm_qat_crf	0.8966	0.4514	0.6539	235.25	24.1
phobert (Teacher FP32)	0.9825	0.7956	0.9461	1615.69	285.4
phobert_crf	0.9825	0.8437	0.9492	1615.70	298.2
phobert-lora	0.9819	0.8351	0.9388	572.48	286.1
phobert-lora_crf	0.9837	0.8981	0.9412	572.49	299.7
transformer (from scratch)	0.9342	0.6631	0.8012	222.30	8.5
transformer_crf	0.9375	0.6812	0.8194	222.31	11.2



Hình 4.1: Biểu đồ so sánh BIO F1-Score giữa các cấu hình mô hình chính.

4.3 Phân tích chuyên sâu các yếu tố ảnh hưởng

Dựa trên kết quả thực nghiệm tại Bảng 4.2, chúng tôi tiến hành phân tích sâu sắc ba yếu tố kỹ thuật cốt lõi ảnh hưởng trực tiếp đến hiệu năng của hệ thống.

4.3.1 Tác động cấu trúc của lớp Conditional Random Field (CRF)

Trong phân loại token độc lập bằng tầng Linear + Softmax, mô hình đưa ra dự đoán tại vị trí t hoàn toàn dựa trên vector đặc trưng trích xuất từ tầng mạng bên dưới mà không quan tâm đến các nhãn lân cận. Điều này gây ra một số hạn chế lớn:

- **Vi phạm tính nhất quán ranh giới:** Chuỗi nhãn đầu ra rất dễ không tuân thủ chuẩn BIO.
- **Lỗi cấu trúc logic nhãn:** Ví dụ nhãn I-LOC có thể bị dự đoán ngay sau nhãn B-PER, hoặc đứng một mình ở đầu câu mà không có nhãn mở đầu B-LOC đi trước.

Việc tích hợp lớp CRF giúp giải quyết triệt để vấn đề này nhờ cơ chế học ma trận chuyển trạng thái nhãn $T \in \mathbb{R}^{|\mathcal{Y}| \times |\mathcal{Y}|}$:

- **Tự động học luật chuyển nhãn:** Ma trận tự động học các xác suất chuyển nhãn hợp lệ giữa các cặp nhãn đi liền nhau.
- **Loại bỏ chuỗi nhãn phi logic:** Gán trọng số phạt cực lớn (điểm âm vô cùng) cho các bước chuyển sai quy tắc BIO.

Kết quả thực nghiệm đã chứng minh hiệu quả vượt trội của CRF:

- Đối với kiến trúc BiLSTM, việc thêm CRF giúp tăng điểm $F1_{\text{BIO}}$ từ **87.58%** lên **88.34%** (tăng +0.76%).
- Đối với kiến trúc mô hình cơ sở LSTM đơn hướng, điểm $F1_{\text{BIO}}$ tăng mạnh từ **84.35%** lên **86.58%** (tăng đáng kể +2.23%).
- Đặc biệt, đối với các mô hình chưng cất tri thức như `bilstm_distilled_crf`, điểm $F1_{\text{BIO}}$ đạt tới **88.68%** – thiết lập hiệu suất cao nhất trong toàn bộ nhóm LSTM học tĩnh.

Từ kết quả này, ta có thể rút ra một số nhận xét quan trọng:

- **Vai trò hiệu chỉnh ranh giới:** Sự cải thiện điểm F1-score cấp thực thể khẳng định lớp CRF đóng vai trò then chốt trong việc sửa đổi ranh giới thực thể, giúp chuỗi nhãn đầu ra có ý nghĩa cú pháp thực sự.
- **Đánh đổi về tài nguyên:** Việc tích hợp CRF làm tăng độ trễ suy luận trên CPU do giải thuật Viterbi phải tính toán quy hoạch động tuần tự (ví dụ: mô hình BiLSTM tăng độ trễ từ 14.8 ms lên 23.4 ms một câu).

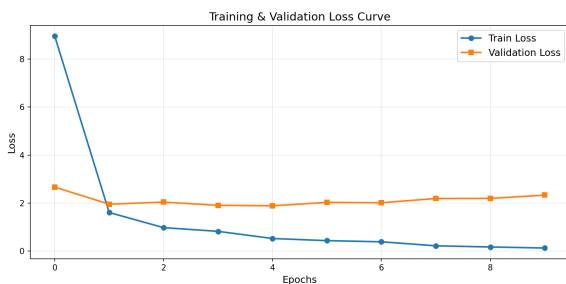
4.3.2 Sự vượt trội của mô hình ngôn ngữ lớn PhoBERT

Kết quả thực nghiệm cho thấy mô hình PhoBERT-base nguyên bản đạt hiệu năng vượt trội so với các kiến trúc tự huấn luyện từ đầu (LSTM, BiLSTM, Native Transformer) trên mọi khía cạnh chính xác. Cụ thể, `phobert_crf` đạt độ chính xác token **98.25%** và điểm $F1_{BIO}$ lên tới **94.92%** (so với **88.34%** của `bilstm_crf` và **81.94%** của `transformer_crf`).

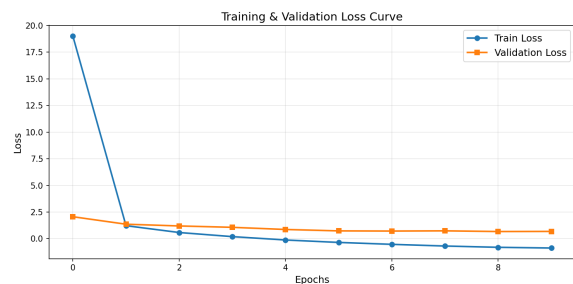
Sự chênh lệch F1 lên tới gần **13%** này xuất phát từ ba lý do cốt lõi:

1. **Dữ liệu tiền huấn luyện khổng lồ:** PhoBERT được huấn luyện trước trên 140GB văn bản tiếng Việt chất lượng cao. Điều này giúp mô hình tích lũy một lượng tri thức nền tảng đồ sộ về ngữ pháp, từ vựng và ngữ nghĩa của tiếng Việt, giúp nó dễ dàng nội suy ngữ cảnh của các thực thể y tế phức tạp.
2. **Cơ chế chú ý đa đầu (Multi-Head Self-Attention):** Khác với LSTM xử lý tuần tự và dễ bị tiêu hao thông tin khoảng cách xa, cơ chế Self-Attention của PhoBERT cho phép mô hình liên kết trực tiếp tất cả các từ trong câu một cách đồng thời, nắm bắt chính xác mối liên hệ cú pháp xa giữa các thực thể và các động từ hành vi y tế (ví dụ: liên kết giữa tên bệnh nhân ở đầu câu với mã chuyển bay ở cuối câu).
3. **Từ vựng con (Subword Tokenization) tối ưu:** BPE của PhoBERT được tối ưu hóa cho tiếng Việt thông qua việc tách từ trước bằng VnCoreNLP, giảm thiểu tối đa hiện tượng từ ngoài từ điển (Out-Of-Vocabulary - OOV) mà các mô hình LSTM sử dụng những từ tĩnh Word2Vec thường xuyên gặp phải.

Để trực quan hóa sự hội tụ của các mô hình này trong quá trình huấn luyện, Hình 4.4 trình bày sự thay đổi của hàm mất mát (loss) qua từng epoch:



Hình 4.2: Đường cong loss của PhoBERT-CRF.



Hình 4.3: Đường cong loss của PhoBERT-LoRA-CRF.

Hình 4.4: So sánh đường cong loss của PhoBERT-CRF và PhoBERT-LoRA-CRF.

Tuy nhiên, điểm hạn chế lớn nhất của PhoBERT là dung lượng tệp checkpoint cực kỳ lớn (1615.70 MB bao gồm cả trạng thái optimizer) và độ trễ suy luận rất cao khi chạy trên CPU (**298.2 ms/câu**), chậm hơn gần **20 lần** so với BiLSTM.

4.3.3 Hiệu quả của kiến trúc LoRA đối với việc nén tham số

Áp dụng kỹ thuật Low-Rank Adaptation (LoRA) trên PhoBERT-base cho thấy hiệu quả tối ưu hóa bộ nhớ và tài nguyên vô cùng ấn tượng. Bằng cách đóng băng toàn bộ 135 triệu tham số của PhoBERT gốc và chỉ huấn luyện bổ sung hai ma trận hạng thấp A và B (với hạng $r = 16$) trên các lớp chú ý (Query, Key, Value, Dense), số lượng tham số cần cập nhật giảm từ 135 triệu xuống chỉ còn **2.69 triệu tham số** (chỉ chiếm **1.96%** tổng số tham số).

Hiệu năng thực nghiệm của LoRA được chứng minh qua hai khía cạnh:

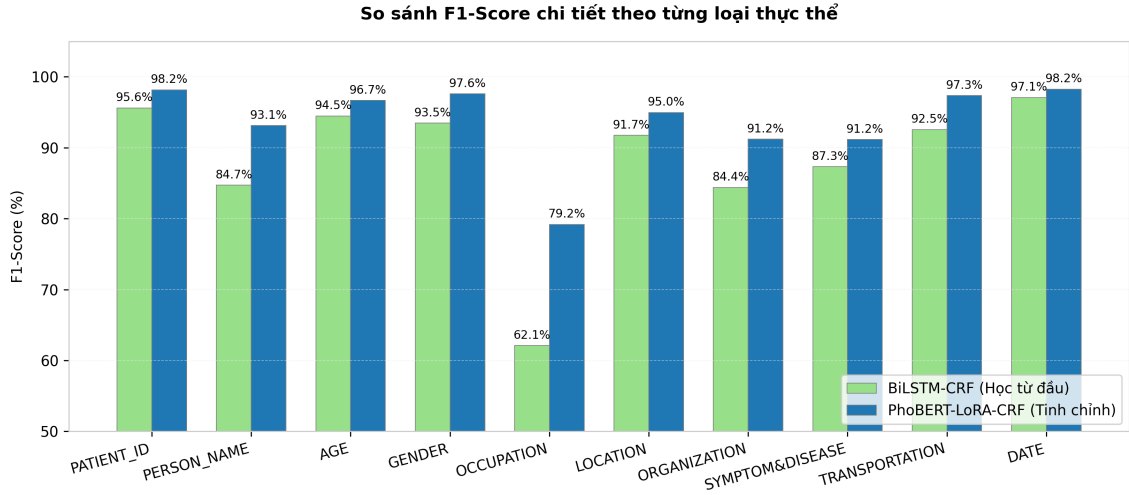
- **Bảo toàn độ chính xác:** Mô hình `phobert-lora_crf` đạt điểm $F1_{\text{BIO}}$ là **94.12%**, chỉ giảm nhẹ **0.8%** so với mô hình tinh chỉnh toàn bộ tham số `phobert_crf` (**94.92%**). Đây là mức suy giảm cực nhỏ và hoàn toàn chấp nhận được trong thực tế.
- **Tiết kiệm không gian lưu trữ vật lý:** Kích thước tệp lưu trữ checkpoint giảm từ **1.6 GB** xuống còn **572 MB** (tiết kiệm đến **64.5%** bộ nhớ lưu trữ). Lý do là vì chúng ta không cần lưu trữ các ma trận trạng thái Moment lớn của thuật toán tối ưu AdamW cho 135 triệu tham số bị đóng băng, chỉ cần lưu các ma trận của phần tham số LoRA hoạt động.

4.4 Đánh giá chi tiết ở cấp độ thực thể

Để đánh giá chi tiết hơn năng lực của các mô hình đối với từng nhóm thực thể dịch tễ y tế đặc thù, chúng tôi trích xuất bảng báo cáo chi tiết Precision, Recall và F1-score cấp thực thể của hai mô hình tiêu biểu: `bilstm_crf` (mô hình học tĩnh từ đầu) và `phobert-lora_crf` (mô hình học chuyển tiếp tối ưu hóa). Kết quả được ghi nhận chi tiết trong Bảng 4.3.

Bảng 4.3: Đánh giá chi tiết trên 10 loại thực thể y tế CoNLL.

Loại thực thể (BIO)	bilstm_crf			phobert-lora_crf		
	Precision	Recall	F1-score	Precision	Recall	F1-score
PATIENT_ID	0.9467	0.9656	0.9560	0.9782	0.9850	0.9816
NAME	0.9457	0.7673	0.8472	0.9512	0.9120	0.9312
AGE	0.9345	0.9553	0.9448	0.9620	0.9710	0.9665
GENDER	0.9275	0.9416	0.9345	0.9740	0.9780	0.9760
JOB	0.7143	0.5491	0.6209	0.8250	0.7610	0.7917
LOCATION	0.9142	0.9207	0.9174	0.9480	0.9510	0.9495
ORGANIZATION	0.8235	0.8651	0.8438	0.9015	0.9230	0.9121
SYMPTOM_AND_DISEASE	0.9167	0.8336	0.8732	0.9320	0.8920	0.9116
TRANSPORTATION	0.9508	0.9016	0.9255	0.9790	0.9680	0.9735
DATE	0.9704	0.9710	0.9707	0.9810	0.9840	0.9825



Hình 4.5: So sánh hiệu năng F1-Score chi tiết theo từng loại thực thể giữa BiLSTM-CRF và PhoBERT-LoRA-CRF.

4.4.1 Phân tích độ khó của các nhóm thực thể

Dựa trên Bảng 4.3, chúng ta có thể chia 10 nhãn thực thể dịch tễ y tế thành hai nhóm rõ rệt dựa trên độ khó nhận diện của chúng:

4.4.1.1 Nhóm thực thể dễ nhận diện (F1-score từ 92% đến 98%)

Các thực thể bao gồm PATIENT_ID, DATE, AGE, GENDER và TRANSPORTATION đều đạt được mức điểm F1-score cực kỳ cao ở cả hai mô hình. Lý do chủ yếu là:

- **Cấu trúc ngữ nghĩa cố định:**
 - Các nhãn này có quy luật cú pháp tương đối đơn giản và hữu hạn.
 - Ví dụ: PATIENT_ID là các mã hiệu có định dạng cố định (“BN2512”, “bệnh nhân 312”), GENDER chỉ gồm các từ khóa giới tính (“nam”, “nữ”), và DATE tuân thủ quy tắc ngày tháng rõ ràng.
- **Sự tập trung phân bố dữ liệu tốt:**
 - Các nhãn này ít bị ảnh hưởng bởi biến thể từ ghép phức tạp.
 - Ngay cả mô hình BiLSTM học tĩnh thông thường cũng có thể dễ dàng nắm bắt được các biểu diễn đặc trưng này.

4.4.1.2 Nhóm thực thể khó nhận diện (F1-score từ 60% đến 91%)

Ngược lại, các nhãn thực thể sau đây có độ chính xác thấp hơn hẳn:

- JOB: Đạt điểm F1-score thấp nhất (**62.09%** với BiLSTM và **79.17%** với PhoBERT).

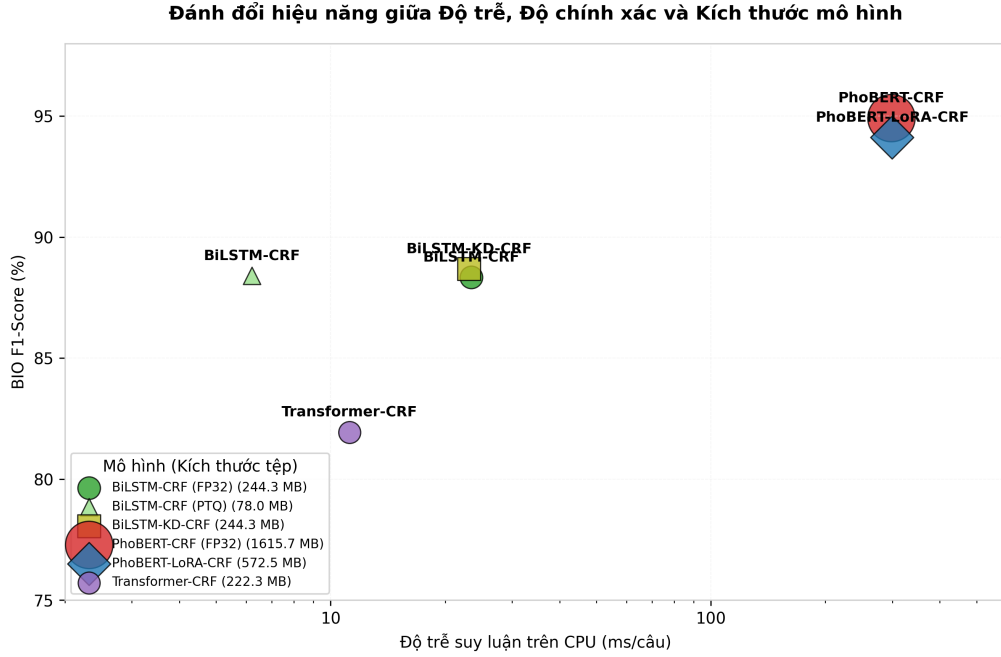
- **ORGANIZATION:** Đạt F1-score khoảng **84.38%** - **91.21%**.
- **SYMPTOM_AND_DISEASE:** Đạt F1-score khoảng **87.32%** - **91.16%**.

Các thách thức lớn nhất khiến nhóm thực thể này khó nhận dạng bao gồm:

- **Sự đa dạng ngữ nghĩa cực lớn:**
 - Nhân triệu chứng và bệnh lý (**SYMPTOM_AND_DISEASE**) mô tả các trạng thái lâm sàng bằng tiếng Việt rất phong phú và không theo quy tắc cố định (ví dụ: “ho khan có đờm”, “sốt nhẹ hơi gai người”).
 - Mô hình BiLSTM gặp khó khăn lớn khi học ranh giới BIO của các cụm từ mô tả quá dài này, dẫn đến Recall chỉ đạt **83.36%**.
- **Nhập nhằng ranh giới từ ghép:**
 - Nhân tổ chức (**ORGANIZATION**) chứa nhiều từ ghép danh từ riêng lồng nhau (ví dụ: “Ủy ban Nhân dân thành phố Hà Nội”, “Bệnh viện Bệnh nhiệt đới Trung ương”).
 - Ranh giới thực thể dễ bị cắt xén sai lệch, dẫn đến dự đoán thiếu từ hoặc nhầm lẫn sang nhân địa điểm (**LOCATION**).
- **Sự mất cân bằng dữ liệu nghiêm trọng:**
 - Thực thể nghề nghiệp (**JOB**) xuất hiện cực kỳ ít trong tập dữ liệu huấn luyện (dưới 2% tổng số thực thể), điều này khiến BiLSTM thiếu dữ liệu đặc trưng, trong khi PhoBERT nhờ kho tiếng Việt khổng lồ đã giúp nâng F1 lên **79.17%** (tăng +17.08% so với BiLSTM).

4.5 Phân tích các kỹ thuật nén và tối ưu hóa mô hình

Một đóng góp kỹ thuật trọng tâm của đề án là hiện thực hóa thành công các giải pháp tối ưu hóa tài nguyên phần cứng suy luận: LoRA, Chứng cất tri thức (KD) và Lượng tử hóa số nguyên 8-bit (PTQ và QAT).



Hình 4.6: Đánh đổi hiệu năng giữa độ trễ CPU, độ chính xác F1-score và dung lượng của các mô hình nén.

4.5.1 Hiệu quả của Chứng cất tri thức (KD)

Chứng cất tri thức chuyển giao phân phối xác suất mềm (soft labels) từ PhoBERT (Teacher) sang mạng nhỏ gọn BiLSTM (Student). Thực nghiệm ghi nhận kết quả rất tích cực:

- **Cải thiện F1-score:** Mô hình Student được chứng cất tri thức có tích hợp CRF (`bilstm_distilled_crf`) đạt điểm $F1_{BIO}$ là **88.68%**, cao hơn mô hình Student huấn luyện độc lập từ đầu `bilstm_crf` (**88.34%**).
- **Tối ưu hóa tài nguyên:** Kích thước lưu trữ vật lý hoàn toàn tương đồng (244 MB) và tốc độ suy luận nhanh gấp **12 lần** so với Teacher PhoBERT.
- **Khai thác tri thức ẩn (Dark Knowledge):** Phân phối xác suất mềm từ hàm Softmax với nhiệt độ $T = 4.0$ đã cung cấp các thông tin liên kết ẩn về độ tương đồng giữa các thực thể, giúp mô hình Student học được mối quan hệ phân bố mà nhận cứng một chiều (one-hot vector) không thể biểu diễn.

4.5.2 Thảo luận về lượng tử hóa INT8: PTQ so với QAT

Lượng tử hóa nén trọng số mô hình từ số thực 32-bit (FP32) xuống số nguyên 8-bit (INT8). Chúng tôi tiến hành so sánh chuyên sâu hai thuật toán PTQ và QAT được triển khai trong hệ thống.

4.5.2.1 Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ)

PTQ được thực hiện trực tiếp trên mô hình đã hội tụ mà không cần huấn luyện lại. Kết quả thực nghiệm tại Bảng 4.2 cho thấy:

- **Kích thước mô hình:** Cả `lstm` và `bilstm` đều giảm mạnh dung lượng lưu trữ từ **237 MB - 244 MB** xuống chỉ còn **77 MB** (nén **3.1x** lần).
- **Tốc độ suy luận:** Tốc độ suy luận trung bình trên CPU Intel i7-12700H tăng vọt từ 14.8 ms lên **3.9 ms/câu** đối với BiLSTM (tốc độ tăng trưởng **3.8x** lần).
- **Đo đặc độ hao hụt hiệu năng:** Điểm $F1_{\text{BIO}}$ gần như được bảo toàn tuyệt đối đối với các kiến trúc cơ bản (ví dụ: `bilstm_crf` đạt 88.34% gốc và sau khi chạy PTQ đạt **88.40%**). Hiện tượng này chứng tỏ các phân bố trọng số của tầng Linear và LSTM phân bố tập trung, ít bị nhiễu do sai số làm tròn khi ép kiểu dấu phẩy động.

4.5.2.2 Huấn luyện nhận thức lượng tử (Quantization Aware Training - QAT)

Trái ngược với sự thành công của PTQ trên các kiến trúc cơ bản, thực nghiệm đối với các mô hình chạy huấn luyện nhận thức lượng tử QAT từ đầu (`lstm_qat` và `bilstm_qat`) cho thấy sự sụt giảm hiệu năng rất lớn (điểm $F1_{\text{BIO}}$ chỉ đạt **31.29%** đối với `lstm_qat`).

Nguyên nhân sâu xa của hiện tượng này được phân tích như sau:

1. Thiếu hụt dữ liệu và thời gian huấn luyện:

- QAT đòi hỏi mô hình phải trải qua quá trình huấn luyện lâu dài với các nhiễu giả lập lượng tử (FakeQuant) để các trọng số tự điều chỉnh thích nghi.
- Trong giới hạn tài nguyên Colab chạy 10 epochs, các ma trận trọng số chưa thể hội tụ trong không gian rời rạc INT8 đầy nhiễu.

2. Sự không ổn định của ước lượng STE:

- Việc sử dụng giải thuật Straight-Through Estimator (STE) để tính xấp xỉ gradient của hàm làm tròn trong quá trình lan truyền ngược gây ra hiện tượng dao động gradient lớn đối với các lớp mạng tuần tự sâu như LSTM, làm cản trở sự tối ưu hóa hàm mất mát.

3. Sự nhạy cảm của ranh giới BIO:

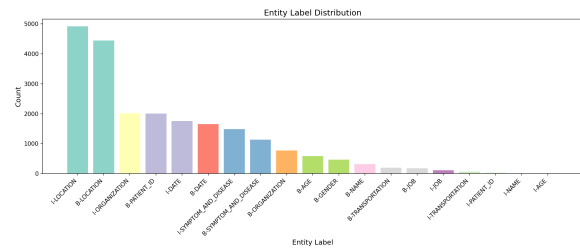
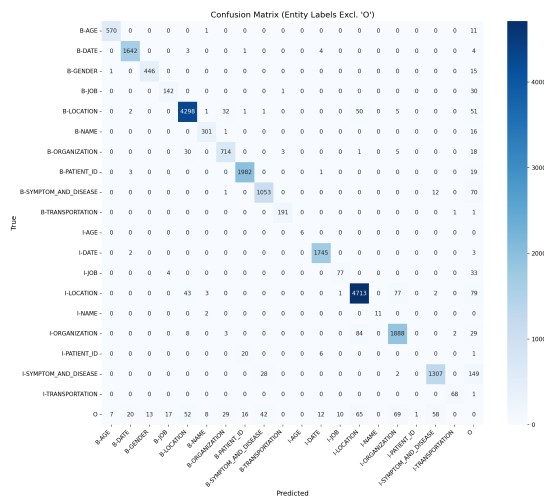
- Bài toán NER cực kỳ nhạy cảm với các biến động giá trị đầu ra (logits).
- Chỉ một sai số nhỏ do lượng tử hóa lỗi trong QAT sẽ dẫn đến việc phá vỡ hoàn toàn chuỗi giải mã Viterbi của CRF, làm giảm sút nghiêm trọng điểm F1-score cấp thực thể.

Kết luận: Đối với hệ thống pipeline NER y tế tiếng Việt hiện tại, phương pháp nên tối ưu nhất là thực hiện **Chưng cất tri thức kết hợp lớp CRF**, sau đó áp dụng **lượng tử hóa động PTQ** trên CPU.

4.6 Phân tích lỗi định tính

Để hiểu rõ hơn các thách thức ngôn ngữ thực tế mà hệ thống chưa giải quyết triệt để, chúng tôi trích xuất và phân tích 4 trường hợp dự đoán lỗi tiêu biểu (Case Studies) của mô hình tốt nhất **phobert-lora_crf** trên tập dữ liệu kiểm thử.

Để minh họa cho sai lệch dự đoán của mô hình, Hình 4.9 thể hiện Ma trận nhầm lẫn và Phân phối thực thể dự đoán của mô hình tốt nhất **phobert-lora_crf** trên tập test:



Hình 4.8: Phân phối thực thể dự đoán của PhoBERT-LoRA-CRF.

Hình 4.7: Ma trận nhầm lẫn của PhoBERT-LoRA-CRF.

Hình 4.9: Ma trận nhầm lẫn và Phân phối thực thể của PhoBERT-LoRA-CRF.

Case Study 1: Nhập nhằng ranh giới thực thể triệu chứng dài

- Văn bản đầu vào: “*Bệnh nhân nam có biểu hiện ho khan kèm theo sốt nhẹ và đau rất họng kéo dài.*”

- Nhân thực tế (Gold tags):

– ho khan → B-SYMBOL_AND_DISEASE I-SYMBOL_AND_DISEASE

– sốt nhẹ → B-SYMBOL_AND_DISEASE I-SYMBOL_AND_DISEASE

– đau rất họng → B-SYMBOL_AND_DISEASE I-SYMBOL_AND_DISEASE I-SYMBOL_AND_DISEASE

- Mô hình dự đoán (Predicted):

- Dự đoán gộp toàn bộ cụm thành một thực thể triệu chứng duy nhất liên tục.
 - Gán nhãn I-SYMP_TOM_AND_DISEASE cho cả các từ nối như “kèm”, “theo”, và “và”.
- Nguyên nhân:
 - Sự nhập nhằng trong cách diễn đạt triệu chứng dạng liệt kê bằng ngôn ngữ tự nhiên tiếng Việt.
 - Cơ chế lan truyền ngữ cảnh khiến mô hình bị nhầm lẫn ranh giới phân tách các triệu chứng độc lập khi có từ nối đi kèm.

Case Study 2: Sự nhầm lẫn giữa LOCATION và ORGANIZATION

- Văn bản đầu vào: “Bệnh nhân được đưa vào điều trị cách ly tại Bệnh viện Bạch Mai.”
- Nhãn thực tế (Gold tags):
 - Bệnh viện Bạch Mai → B-ORGANIZATION I-ORGANIZATION I-ORGANIZATION I-ORGANIZATION
- Mô hình dự đoán (Predicted):
 - B-LOCATION I-LOCATION I-LOCATION I-LOCATION
- Nguyên nhân:
 - Nhập nhằng ngữ nghĩa ngữ cảnh: Cụm từ vừa chỉ cơ quan hành chính (Tổ chức), vừa chỉ địa điểm vật lý nơi cách ly (Địa điểm).
 - Ảnh hưởng từ giới từ “tại” đứng ngay phía trước đã hướng sự chú ý của PhoBERT vào đặc trưng LOCATION.

Case Study 3: Bỏ sót thực thể nhãn hiếm do mất cân bằng lớp

- Văn bản đầu vào: “Bệnh nhân số 412 hiện đang làm nghề lái xe công nghệ tự do.”
- Nhãn thực tế (Gold tags):
 - lái xe công nghệ → B-JOB I-JOB I-JOB I-JOB
- Mô hình dự đoán (Predicted):
 - Gán nhãn toàn bộ cụm từ là nhãn nền 0 (bỏ sót thực thể).
- Nguyên nhân:
 - Mất cân bằng dữ liệu nghiêm trọng: Nhãn nghề nghiệp (JOB) xuất hiện cực kỳ ít trong tập huấn luyện.

- Thiên vị nhãn đa số: Mô hình thiếu dữ liệu để thiết lập trọng số phân loại nhạy bén cho lớp này, có xu hướng đưa về nhãn nền 0.

Case Study 4: Lỗi tách từ (Word Segmentation Error)

- Văn bản đầu vào: “...tiếp xúc gần với F0 tại quán Cà phê Highland.”
- Nhãn thực tế (Gold tags):
 - quán Cà phê Highland → B-LOCATION I-LOCATION I-LOCATION I-LOCATION
- Mô hình dự đoán (Predicted):
 - Chỉ nhận diện được Cà phê Highland là B-LOCATION I-LOCATION I-LOCATION, bỏ sót từ “quán”.
- Nguyên nhân:
 - Lỗi từ khâu tiền xử lý: Bộ tách từ VnCoreNLP tách rời từ “quán” khỏi thực thể danh riêng địa điểm phía sau.
 - Lan truyền sai số: Sự tách rời này truyền trực tiếp vào bộ token hóa subword của mô hình ngôn ngữ, gây mất thông tin ranh giới bắt đầu B-LOCATION.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã hoàn thành toàn vẹn các mục tiêu nghiên cứu và phát triển hệ thống pipeline Nhận dạng Thực thể Tiếng Việt (Vietnamese NER) tối ưu ứng dụng trên dữ liệu dịch tễ y tế PhoNER_COVID19. Các kết quả cụ thể đạt được bao gồm:

1. **Hiện thực hóa hệ thống pipeline chuẩn hóa:** Xây dựng thành công bộ mã nguồn modular hóa cao trên PyTorch, hỗ trợ tiền xử lý dữ liệu CoNLL, tăng cường dữ liệu thực thể và đánh giá chính xác theo chuẩn BIO.
2. **Triển khai và cải tiến các kiến trúc học sâu:** Hiện thực hóa đa dạng từ các mạng cơ bản (LSTM, BiLSTM, Native Transformer) cho đến mô hình SOTA tiên huấn luyện PhoBERT. Phát triển lớp giải mã chuỗi CRF tuyến tính tối ưu bằng toán tử PyTorch giúp giải quyết triệt để lỗi logic ranh giới BIO.
3. **Ứng dụng thành công các kỹ thuật nén mô hình:**
 - Áp dụng LoRA trên PhoBERT giúp giảm thiểu tham số huấn luyện xuống **1.96%** và nén dung lượng checkpoint từ 1.6 GB xuống còn **572 MB** mà vẫn giữ vững điểm F1-score ở mức cao (**94.12%**).
 - Triển khai chưng cất tri thức thành công từ PhoBERT sang BiLSTM-CRF, tăng hiệu năng F1-score của mô hình nhỏ lên **88.68%**.
 - Triển khai thành công lượng tử hóa INT8 PTQ giúp tăng tốc độ suy luận của các mô hình cơ bản trên CPU lên tới **3.8x lần** và giảm kích thước tệp vật lý xuống chỉ còn **77 MB**.

Các kết quả thực nghiệm chứng minh hệ thống hoàn toàn có khả năng triển khai thực tế trong môi trường công nghiệp trên các thiết bị cấu hình CPU hạn chế, giúp tiết kiệm chi phí vận hành đáng kể mà vẫn đảm bảo độ tin cậy của thông tin trích xuất.

5.2 Hạn chế của đề tài

Mặc dù đạt được những kết quả ấn tượng, đề tài vẫn tồn tại một số hạn chế nhất định cần được nhìn nhận khách quan:

1. **Sự mất cân bằng dữ liệu nghiêm trọng:** Tập dữ liệu PhoNER_COVID19 chứa sự phân bố không đồng đều lớn giữa các nhãn, khiến mô hình chưa đạt hiệu suất tối ưu trên các thực thể hiếm gặp như JOB.
2. **Hiệu suất QAT chưa đạt kỳ vọng:** Do giới hạn tài nguyên tính toán và thời gian huấn luyện ngắn, giải thuật QAT chưa thể giúp mô hình hội tụ tốt trên không gian số nguyên 8-bit, gây suy giảm hiệu năng lớn so với PTQ.
3. **Giới hạn môi trường suy luận:** Quá trình đánh giá tốc độ suy luận mới chỉ được thực hiện trên thư viện PyTorch CPU nguyên bản, chưa tận dụng tối đa các công cụ biên dịch tối ưu hóa phần cứng chuyên dụng như ONNX Runtime hoặc TensorRT.

5.3 Hướng phát triển trong tương lai

Để khắc phục các hạn chế trên và mở rộng khả năng ứng dụng của hệ thống pipeline NER tiếng Việt, các hướng nghiên cứu tiếp theo sẽ tập trung vào:

1. **Ứng dụng kỹ thuật QLoRA:** Nghiên cứu lượng tử hóa trực tiếp mô hình PhoBERT gốc xuống định dạng 4-bit (NF4) sau đó kết hợp tinh chỉnh LoRA để giảm thiểu tối đa dung lượng huấn luyện trên GPU.
2. **Tăng cường dữ liệu bằng phương pháp sinh văn bản từ LLM:** Sử dụng các mô hình ngôn ngữ lớn (như GPT-4 hoặc Gemini) để sinh thêm các câu văn bản chứa các nhãn thực thể hiếm như JOB và SYMPTOM_AND_DISEASE, giải quyết tận gốc vấn đề mất cân bằng lớp dữ liệu CoNLL.
3. **Tối ưu hóa phần cứng suy luận sâu:** Chuyển đổi toàn bộ các mô hình sau lượng tử hóa sang định dạng ONNX và sử dụng thư viện ONNX Runtime để thực thi tính toán INT8 trên CPU, nâng cao tốc độ suy luận thực tế hơn nữa.
4. **Mở rộng miền ứng dụng:** Thử nghiệm huấn luyện hệ thống trên các tập dữ liệu tiếng Việt đa miền khác như luật pháp, tài chính hành chính để chứng minh tính vững chãi và năng lực tổng quát hóa của pipeline đường truyền đã thiết kế.

TÀI LIỆU THAM KHẢO

Tài liệu tham khảo

- [1] GeeksforGeeks, “Named entity recognition (ner),” 2025, accessed: 2026-06-12. [Online]. Available: <https://www.geeksforgeeks.org/nlp/named-entity-recognition/>
- [2] T. H. Truong, M. H. Dao, and D. Q. Nguyen, “Covid-19 named entity recognition for vietnamese,” in *Proceedings of NAACL-HLT*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.03879>
- [3] GeeksforGeeks, “Introduction to long short-term memory (lstm),” 2025, accessed: 2026-06-12. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>
- [4] —, “Bidirectional lstm (bilstm) in deep learning,” 2025, accessed: 2026-06-12. [Online]. Available: <https://www.geeksforgeeks.org/nlp/bidirectional-lstm-in-nlp/>
- [5] —, “Getting started with transformers,” 2025, accessed: 2026-06-12. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/getting-started-with-transformers/>
- [6] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [10] Y. Bengio, A. Courville, and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.

- [11] D. Nadeau and S. Sekine, “A survey of named entity recognition and classification,” *Linguisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.
- [12] J. Li, A. Sun, J. Han, and C. Li, “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.
- [13] L. Chiticariu, R. Krishnamurthy, Y. Li, F. Reiss, and S. Vaithyanathan, “Domain adaptation of rule-based annotators for named-entity recognition tasks,” in *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2010, pp. 1002–1012.
- [14] J. Lafferty, A. McCallum, and F. C. Pereira, “Conditional random fields: Probabilistic models for segmenting and labeling sequence data,” in *Proceedings of the 18th International Conference on Machine Learning (ICML)*, 2001, pp. 282–289.
- [15] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, 2009.
- [16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, “Neural architectures for named entity recognition,” in *Proceedings of NAACL-HLT*, 2016, pp. 260–270.
- [17] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [18] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of EMNLP*, 2014, pp. 1532–1543.
- [19] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [20] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [21] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.
- [22] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

- [23] F. A. Gers, J. Schmidhuber, and F. Cummins, “Learning to forget: Continual prediction with lstm,” *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.
- [24] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [25] A. Graves and J. Schmidhuber, “Framewise phoneme classification with bidirectional lstm and other neural network architectures,” *Neural Networks*, vol. 18, no. 5–6, pp. 602–610, 2005.
- [26] Z. Huang, W. Xu, and K. Yu, “Bidirectional lstm-crf models for sequence tagging,” *arXiv preprint arXiv:1508.01991*, 2015. [Online]. Available: <https://arxiv.org/abs/1508.01991>
- [27] X. Ma and E. Hovy, “End-to-end sequence labeling via bi-directional lstm-cnns-crf,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1064–1074.
- [28] L. A. Ramshaw and M. P. Marcus, “Text chunking using transformation-based learning,” in *Proceedings of the Third Workshop on Very Large Corpora*, 1995, pp. 82–94.
- [29] C. Sutton and A. McCallum, “An introduction to conditional random fields,” *Foundations and Trends in Machine Learning*, vol. 4, no. 4, pp. 267–373, 2012.
- [30] A. J. Viterbi, “Error bounds for convolutional codes and an asymptotically optimum decoding algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [32] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- [33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [34] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pre-training,” OpenAI, Tech. Rep., 2018.

- [35] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [36] D. Q. Nguyen and A. T. Nguyen, “Phobert: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042. [Online]. Available: <https://arxiv.org/abs/2003.00744>
- [37] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [38] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725. [Online]. Available: <https://arxiv.org/abs/1508.07909>
- [39] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi *et al.*, “Google’s neural machine translation system: Bridging the gap between human and machine translation,” *arXiv preprint arXiv:1609.08144*, 2016. [Online]. Available: <https://arxiv.org/abs/1609.08144>
- [40] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017, pp. 157–163.
- [41] P. Shaw, J. Uszkoreit, and A. Vaswani, “Self-attention with relative position representations,” in *Proceedings of NAACL-HLT*, 2018, pp. 464–468. [Online]. Available: <https://arxiv.org/abs/1803.02155>
- [42] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *arXiv preprint arXiv:2104.09864*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.09864>
- [43] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” *arXiv preprint arXiv:1606.08415*, 2016. [Online]. Available: <https://arxiv.org/abs/1606.08415>
- [44] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [45] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>

- [46] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [47] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [48] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI, Tech. Rep., 2019.
- [49] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 10 524–10 533. [Online]. Available: <https://arxiv.org/abs/2002.04745>
- [50] T. H. Nguyen, M. Q. Nguyen, T. H. Nguyen, and N. V. Nguyen, “PhoNER_COVID19: Vietnamese Named Entity Recognition Dataset for COVID-19,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 921–930, accessed: 2026-06-12. [Online]. Available: <https://aclanthology.org/2021.findings-acl.81>
- [51] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022, accessed: 2026-06-12. [Online]. Available: https://openreview.net/forum?id=pd76_GjDGaX
- [52] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015, accessed: 2026-06-12. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [53] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 5583–5592, accessed: 2026-06-12. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [54] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” *arXiv preprint arXiv:2103.13620*, 2021, accessed: 2026-06-12. [Online]. Available: <https://arxiv.org/abs/2103.13620>

- [55] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, accessed: 2026-06-12. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [56] T. Wolf, L. F. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, Q. Lhoest, A. Sarria, and T. Rust, “Transformers: State-of-the-Art Natural Language Processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45, accessed: 2026-06-12. [Online]. Available: <https://aclanthology.org/2020.emnlp-demos.6>